

FOODSTORY

From Responsive Vue Prototype to Full-Stack, Real-Time and Vision-Assisted Food Discovery

Custom Web Application Project Report

COS30043 - Interface Design and Development



Figure 1. Progressive development from the Stage 1 Vue foundation to the Stage 3 advanced technique.

Student	Nguyen Hoang Long
Student ID	105556692
Project	FoodStory
Advanced technique	Advanced Vue composable for vision-assisted food place discovery
Final branch	codex/render-deployment
Final source commit	0473287788bad6a098f1d2724ec0e3d0c81bd412
Report date	27 July 2026

Archive-Verified Report - Stage 3 Video URL Placeholder - commit 04732877

Academic Integrity and AI-Assistance Statement

This report and application are the submission for which I accept responsibility. I used ChatGPT and Codex more deeply than proofreading: they proposed design alternatives, code edits, test cases and report revisions, and some multi-file changes were applied through AI-assisted workflows. I inspected the resulting diffs, ran the affected application flows and tests, kept, revised or discarded changes, and prepared to explain the final implementation orally. Generated text or code is not treated as evidence by itself; claims are tied to executable source, observed behaviour, test output or an explicitly stated limitation. Provider-dependent demonstrations remain separated from deterministic fixtures and historical evidence.

The technical source authority for this report is commit 0473287788bad6a098f1d2724ec0e3d0c81bd412 on branch codex/render-deployment. The public production build is available at <https://foodstory-cos30043-hoanglong212.onrender.com>. Author-run final verification records frontend npm ci passed, 48/48 frontend tests passed, the frontend production build passed, backend npm ci passed, 399/399 backend tests passed and a clean Git worktree at the authority gate. These results are not an independent external certification. The separate source archive FoodStory_Final_Source_04732877.zip contains 814 entries (767 files and 47 directories), is 55,496,926 bytes (52.925993 MiB), and has SHA-256 3d66a5f34ac2bbc854c0ce4104c89a85a523385b33216bf4cc60b5d0d2d23e1d. ZIP integrity, every-entry reads, safe paths, forbidden/sensitive-file and likely-secret scans, clean extraction and expected project structure all passed. The report-input DOCX was excluded from the source archive. The Stage 3 video URL remains [STAGE_3_VIDEO_URL_TO_BE_INSERTED].

Executive Summary

FoodStory is a progressively developed Vue 3 web application that combines food journalism, recipe discovery, authenticated community participation, grounded conversational assistance, personal geospatial memories and vision-assisted food discovery. The project began as the required Stage 1 Home, News and About application. It then evolved into a full-stack platform with JWT authentication, MySQL persistence, recipe search, comments, ratings, favourites, shopping checklists, user submissions, moderation, real-time WebSocket updates and FoodBot, a bilingual local-data-first assistant with bounded recent-turn context. Stage 3 introduced an interactive Leaflet Food Map and the assessed advanced Vue technique: a reusable composable that manages long-running, cancellable Vision Auto jobs and safely synchronises structured outcomes with the map.

The report triangulates the final source snapshot, automated tests, API and database evidence, two-client real-time acceptance, controlled historical benchmarks, final Render browser evidence and development history. The final commit passed 48/48 frontend tests, the frontend production build and 399/399 backend tests, with a clean worktree. These final totals come from author-run local commands rather than an independent CI service. Earlier focused API, WebSocket, Vision, Guardian, AI-service and database runs are retained as supplementary historical evidence by concern; they overlap with broader suites and are not added together or relabelled as final-commit totals. Informal usability feedback led to focused recipe search, sticky Recipe Detail navigation, collapsible content, bounded comment loading, reduced motion and address-assisted Food Map placement. The deployment also demonstrates Geoapify and OpenStreetMap-backed serving-place alternatives after user-confirmed dish recognition.

The strongest engineering contribution is the Vue orchestration around uncertain, long-running work: reactive state, cancellation, stale-run authority, lifecycle cleanup, user confirmation and Leaflet synchronisation. The final revision fixed a late create response that could contaminate a newer run and a polling delay that survived cancellation. The product also changed direction when experiments showed that exact restaurant recovery was often unsupported; the dish-first contract preserves usefulness without presenting nearby alternatives as the filming location. This report evaluates orchestration safety and failure handling more strongly than recognition accuracy. It does not claim a representative dish top-1/top-3 or exact-place accuracy result because a labelled real-video benchmark was not completed.

Table of Contents

- Academic Integrity and AI-Assistance Statement
- Executive Summary
- 1. Project Context, Motivation and Objectives
- 2. Methodology, Evidence and Evaluation Framework
- 3. Stage 1 - Establishing the Responsive Vue Foundation
- 4. Stage 2 - Evolving into a Full-Stack Interactive Platform
- 5. Stage 3 - Advanced Vue Composable and Vision-Assisted Food Discovery
- 6. Integrated Testing and Evaluation
- 7. Security, Accessibility, Privacy and Responsible AI
- 8. Critical Discussion and Personal Reflection
- 9. Future Development
- 10. Conclusion
- References
- Appendices A-F

List of Figures

- Figure 1. Progressive development from the Stage 1 Vue foundation to the Stage 3 advanced technique.
- Figure 2. Evidence triangulation used to support implementation and evaluation claims.
- Figure 3. FoodStory Home page in light and dark modes.
- Figure 3A. Final Live Food News interface on the Render deployment.
- Figure 4. Required About interaction and Food Mood enhancement.
- Figure 5. Registration validation, login and mobile navigation.
- Figure 6. Current full-stack architecture and optional provider boundaries.
- Figure 7. Recipe discovery catalogue and detailed interaction surface.
- Figure 7A. Final recipe UX refinement: focused search, sticky guide, progressive disclosure and bounded comments.
- Figure 8. Persist-before-broadcast real-time sequence.
- Figure 9. Two-client comment propagation without refresh.
- Figure 10. Administrator dashboard and moderation queue.
- Figure 10A. User recipe submission and the same record in the Admin pending queue.
- Figure 10B. Administrator approval and the resulting public recipe page.
- Figure 10C. FoodBot grounded answer with a FoodStory source card.
- Figure 10D. Bounded FoodBot conversational follow-up using recent recipe context.
- Figure 11. Food Map overview and personal food memory.
- Figure 11A. Progressive Food Map access for guests and authenticated users.
- Figure 11B. Address-assisted place creation and dish-to-place discovery in the final Food Map.
- Figure 12. Evolution from image OCR to dish-first discovery.
- Figure 13. Reactive state machine managed by useVisionAuto.
- Figure 13A. Active Vision Auto analysis and reusable state after cancellation.
- Figure 14. Stage 3 sequence across Vue, job API, coordinator and worker.
- Figure 15. Track 2 V3 versus Final router safety benchmark.
- Figure 16. Test-to-concern matrix for the advanced feature.
- Figure 17. Vision Auto user-facing progress and uncertainty states.
- Figure 17A. Historical provider-unavailable boundary retained as failure-state evidence.
- Figure 17B. Live Geoapify and OpenStreetMap-backed serving-place alternative in the final deployment.
- Figure 18. Pre-remediation build/output evidence and final asset remediation.

- Figure 19. Home and route-level LCP comparison.
- Figure 20. API median and tail-latency comparison.
- Figure 21. Real-time latency across operations and viewer counts.
- Figure 22. Responsive issue changes from Stage 2 to Final.

- Figure 22A. Visible keyboard focus on the Skip to main content control.
- Figures 23-26. Final-functionality supplemental browser evidence.

List of Tables

- Table 1. Project objectives and measurable evidence.
- Table 2. Evidence labels used in this report.

Evidence independence. Most measurements in this report are reproducible author-run project checks rather than independent verification. The public Render deployment allows direct interface inspection, while the final source archive, checksum, integrity result and sensitive-file scan are current packaging evidence for commit 0473287788bad6a098f1d2724ec0e3d0c81bd412. This distinction is stated explicitly so that pass counts are read as project evidence, not external certification.

- Table 3. Stage 1 technology comparisons.
- Table 4. Stage 1 requirement evaluation.
- Table 5. Stage 2 role model.
- Table 6. Stage 2 technology decisions.
- Table 7. Stage 2 requirement evaluation.
- Table 8. Vision Auto evolution timeline.
- Table 8A. Public contract of useVisionAuto.
- Table 8B. Architectural alternatives for long-running Vue state.
- Table 8C. Controlled Stage 3 router benchmark.
- Table 9. Stage 3 technology decisions.
- Table 10. Final verification results.
- Table 11. Quality-attribute evaluation.
- Table 12. Priority future work.
- Tables 13-16. Appendix rubric, technology, test and failure registers.

1. Project Context, Motivation and Objectives

1.1 Problem Context

Food discovery is fragmented across recipe websites, editorial food news, social-media videos, map applications and personal notes. A person may see an unfamiliar dish in a short video, search separately for its name, open another service to locate restaurants, and then lose the original context. Conventional recipe sites solve only part of this journey: they explain how to cook, but rarely connect the recipe, place, memory, community and visual discovery into one persistent experience.

FoodStory addresses this fragmentation through a single food-centred application. It combines public discovery with personalised and social functions. Guests can understand the concept, read current food news and browse recipes without being forced to register. Authenticated users can rate, comment, save favourites, create shopping checklists, contribute recipes and record personal places. Administrators receive additional governance tools. The Food Map converts food experiences into a spatial personal history, while Vision Auto investigates whether unstructured public video can be transformed into useful, reviewable discovery data.

The project initially focused on Vietnamese cuisine and Ho Chi Minh City. The red visual identity was personally selected because of its connection to the Vietnamese flag; white and warm neutral surfaces were used to create contrast and keep food photography dominant. This cultural starting point does not restrict the long-term scope: the current dish-recognition direction is designed to support food from multiple countries.

1.2 Personal Motivation

This was my first substantial project in Vue. Before COS30043 I preferred React, so the initial difficulty was not basic HTML or CSS but changing my mental model. I had to remember when Vue expected ref, computed, watchers, lifecycle functions and template directives rather than React-style hooks and dependency arrays. FoodStory became a practical way to learn Vue through an application I genuinely wanted to use rather than through disconnected exercises.

I also wanted the project to look different from a typical student CRUD application. The early Home page, editorial typography, dark mode and large food images established that ambition. I repeatedly polished the design after discussing ideas with AI tools, but I retained responsibility for selecting the domain, colour identity, layout, interaction and final implementation. The result was not designed from one complete Figma specification; it evolved through iterative coding, browser review and revision.

1.3 Aim and Objectives

The aim was to develop an extensible Vue application that moves from public food discovery to persistent community interaction and finally to responsible vision-assisted map discovery.

Objective	Implementation intent	Primary evidence
Vue foundation	Implement the required Vue 3, Vite, Vue Router, Home, News and About features with distinct desktop, tablet and mobile layouts.	Stage 1 source, historical JSON, current screenshots and frontend tests.

Objective	Implementation intent	Primary evidence
Full-stack functionality	Add authentication, role-aware access, recipe search, social functions, authorised CRUD and persistent relational data.	API routes, database schema, Stage 2 screenshots and an earlier 87/87 focused API run (historical; not rerun after the final UX/provider refinement).
Real-time interaction	Allow users on different accounts to observe comment and rating updates without refreshing.	WebSocket source, database constraints and an earlier 4/4 two-client acceptance run (historical; not rerun after the final UX/provider refinement).
Personal Food Map	Allow users to record and revisit food places as part of their personal FoodStory.	Leaflet source, personal food-spot persistence and map screenshots.
Advanced Vue technique	Encapsulate long-running Vision Auto orchestration in a reusable composable with cancellation and stale-run protection.	useVisionAuto source, final frontend regression coverage and earlier focused reliability/adaptor suites retained as historical risk evidence.
Responsible AI behaviour	Avoid converting weak visual evidence or provider candidates into false restaurant claims.	Outcome contracts, safe not-found states, historical provider-failure evidence and live Geoapify/OpenStreetMap alternatives labelled as reviewable suggestions.
Grounded conversational discovery	Provide bilingual recipe/place assistance through local-data-first routing, bounded recent context and source-labelled responses, with visual search isolated behind an AI-service boundary.	FoodBot source and tests, current-commit grounded/memory screenshots, and security-boundary smoke checks.

Table 1. Project objectives and measurable evidence.

1.4 Scope

The final application scope includes the Vue frontend, Express API, MySQL/MariaDB schema, WebSocket service, FoodBot routing and bounded browser memory, local and external providers, FastAPI AI service, database migrations, deterministic tests, address geocoding and evidence documentation. For assessment, useVisionAuto and its integration with FoodMapView and Leaflet constitute the Stage 3 advanced Vue technique. Express job coordination, OCR/ASR, provider adapters, FastAPI and FoodBot are supporting or complementary components, discussed where they shape the interface contract, lifecycle, security or evidence. The final public deployment is available on Render at <https://foodstory-cos30043-hoanglong212.onrender.com>. The Stage 3 face-present video remains a separate final submission action; Mercury-specific compatibility should be confirmed only if the unit requires that platform explicitly.

Central research question

How can a Vue application coordinate public content, persistent multi-user interaction and uncertain vision-assisted discovery without sacrificing usability, consistency or trust?

2. Methodology, Evidence and Evaluation Framework

2.1 Incremental Development Method

The assignment required each stage to extend the existing application rather than create a new project (COS30043 Project Brief, 2026). FoodStory followed this constraint. Stage 1 established the route and visual foundation; Stage 2 added application state, server logic and persistence; Stage 3 reused the same authenticated application and Food Map as the interaction surface for the advanced technique. This continuity is important because the later complexity is evidence of architectural evolution, not an unrelated demonstration.

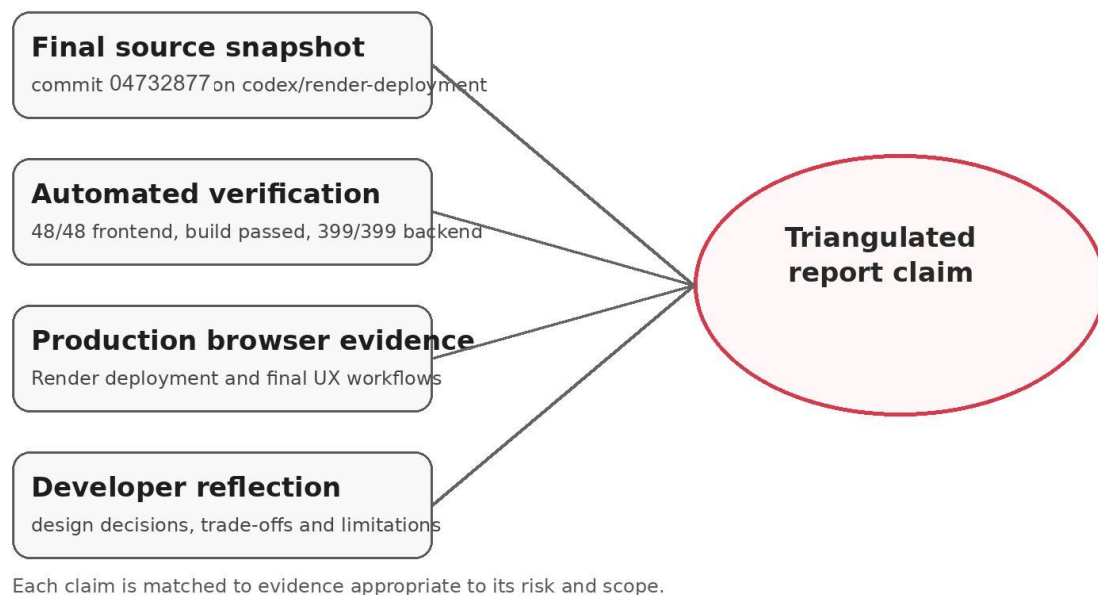


Figure 2. Evidence triangulation used to support implementation and evaluation claims.

2.2 Evidence Authority

The report uses commit 0473287788bad6a098f1d2724ec0e3d0c81bd412 on branch codex/render-deployment as the primary technical authority. Source inspection establishes implementation; the configured final npm commands establish the 48/48 frontend and 399/399 backend baselines; and final Render screenshots establish the deployed interface. These command results were produced locally by the author and are not presented as independent CI, penetration testing or third-party certification. Earlier focused API, WebSocket, Vision, Guardian, AI-service, database and performance runs are dated supplementary evidence only; their potentially overlapping counts are not added to the configured totals. Final packaging evidence comes only from FoodStory_Final_Source_04732877.zip and its current SHA-256 and validation results.

The evidence package also records failed first attempts. For example, the first frontend lock validation found missing lock entries, the first real-time acceptance fixture omitted a required image URL, and the first database audit encountered an old migration-ledger format. Each issue was diagnosed, corrected and rerun. Retaining these failures strengthens the report because it demonstrates

verification and corrective action rather than presenting an artificial history in which every command passed immediately.

2.3 Evidence Labels

Status	Meaning in this report
Live current UI	Captured from the final Render deployment using real routes and the final source branch.
Live database interaction	A temporary record was created through the actual UI/API and later cleaned.
Two-browser acceptance	Two independent authenticated contexts observed a WebSocket update without refresh.
Deterministic fixture	The current interface rendered a bounded test response; it is not a paid-provider success claim.
Source-verified	The behaviour exists in source and tests but was not destructively repeated in the shared environment.
Historical evidence	An earlier Stage 1 implementation retained to show progression; it is not the current source.

Table 2. Evidence status vocabulary.

2.4 Comparative Evaluation

Technology comparisons are project-specific. A library is not judged as universally superior because it is popular. Each decision is evaluated against the assignment requirement, FoodStory’s actual domain, implementation complexity, maintainability, security, testability and future scale. For example, WebSocket is justified for immediate recipe-room updates but not for every request; local JSON is appropriate historical Stage 1 evidence but not the final news source; and an in-memory job queue is defensible for one academic process but not for multi-instance production. The controlled comparative benchmark uses feature-specific Git snapshots rather than one universal baseline. It preserves individual observations, applies warm-up and repeated measurements where supported, keeps unavailable comparisons blank, and separates application-only measurements from external-provider latency (FoodStory, 2026c).

2.5 Requirement Status

Requirement tables use qualitative status labels rather than self-awarded percentages. “Implemented” means the requirement is present and directly evidenced; “implemented with limitations” identifies a bounded gap; “formative evidence only” indicates a small or informal evaluation; and “not formally evaluated” identifies missing structured evidence. These labels describe the submitted evidence and are not predicted marks.

3. Stage 1 - Establishing the Responsive Vue Foundation

3.1 Stage 1 Requirements and Development Order

Stage 1 required a Vue 3 project created with Vite or Vue CLI, Vue Router, Home, News and About single-file components, local JSON news with search and pagination, a reactive name greeting, a two-radio image choice and three visibly different responsive layouts. I implemented the pages in the order Home, News and About. Home established the visual identity; News concentrated array, search and pagination logic; About was technically smaller but became the hardest design problem because its prescribed form and radio interaction had to feel natural inside a food platform.

3.2 Learning Vue from a React-Oriented Starting Point

Vue's Composition API allows stateful logic to be written with functions such as `ref`, `computed` and lifecycle hooks, and Vue describes a composable as a function that encapsulates reusable stateful logic (Vue.js, 2026a). This was familiar in principle because I had previously preferred React, but the execution model is different. React component functions re-run and hooks are governed by call order and dependency arrays; Vue uses fine-grained reactive dependency tracking. The early challenge was therefore not remembering isolated syntax but learning which values should be refs, which values were derived through computed properties and when side effects required cleanup.

The choice of Vue was fixed by the assignment, but it also fit the interactions well. `v-model` made the About inputs and radio selection explicit in the template, `v-for` rendered news and recipe arrays, and computed properties expressed derived search results. The later `useVisionAuto` composable shows that the same concepts learned in Stage 1 scaled into Stage 3 rather than being discarded.

3.3 Home Page and Visual Identity

The initial Home page was created in one main development cycle and later polished rather than conceptually replaced. Its information architecture includes an editorial hero, introductory copy, featured recipes, daily inspiration, a quotation and topic navigation. The final Home additionally calls TheMealDB for inspiration, but the Stage 1 contribution was the layout, brand direction and route entry points.

Red was selected as a cultural and personal reference to Vietnam, while white and cream surfaces protect readability and food-image contrast. The dark theme adapts surface, border and typography values while preserving the red identity. It is therefore a designed theme rather than a simple colour inversion. The combination of serif headings and sans-serif operational text creates an editorial voice without sacrificing form and navigation clarity.

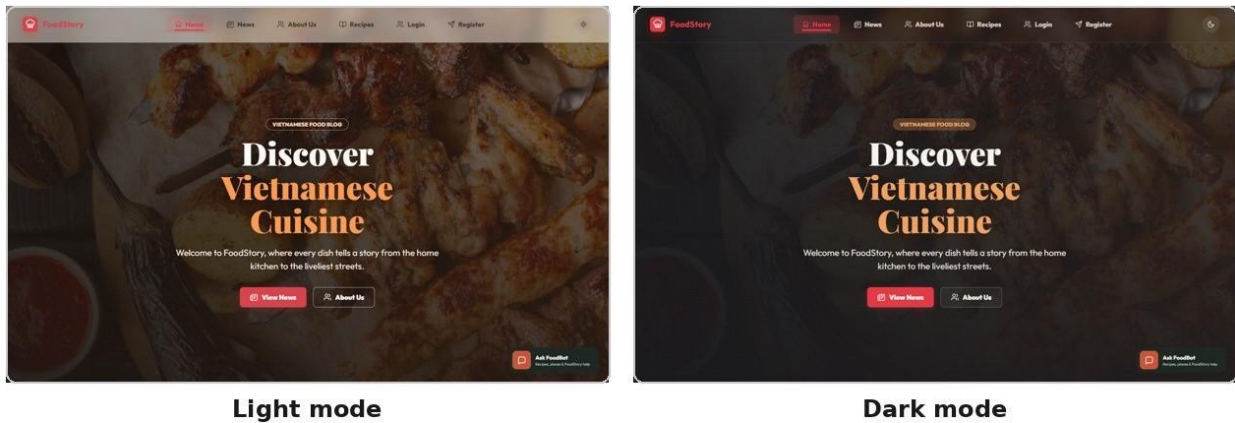


Figure 3. FoodStory Home page in light and dark modes, preserving the same visual hierarchy and brand identity.

3.4 News: Historical JSON and Final API Evolution

The Stage 1 News implementation used a local JSON file containing 12 records. Each item included an identifier, date, title, content and category, with additional display-oriented date fields. Categories included Recipes, Vietnamese Cuisine, Places, Trends, Street Food and Ingredients. Four records were shown per page, which produced enough content to demonstrate pagination. Search covered date, title, content and category as required.

The final application retains the visual and reactive design but replaces the static source with the Guardian Open Platform through an Express backend proxy. News.vue sends page, pageSize, search, date and category parameters to /news/external. The backend stores the key server-side, applies timeout and caching rules, normalises provider records and returns a stable application contract. This evolution must not be used to erase the local JSON evidence: the correct report narrative is that Stage 1 met the JSON requirement and the final project later upgraded the same feature.

Code listing 1. Final News request contract and stale-request cancellation.

The final Render interface preserves the Guardian attribution while presenting keyword, date and category controls above the article stream. This deployed view is stronger evidence than source-only API claims because it demonstrates the final visual hierarchy, external-provider labelling and live content contract in the same route.

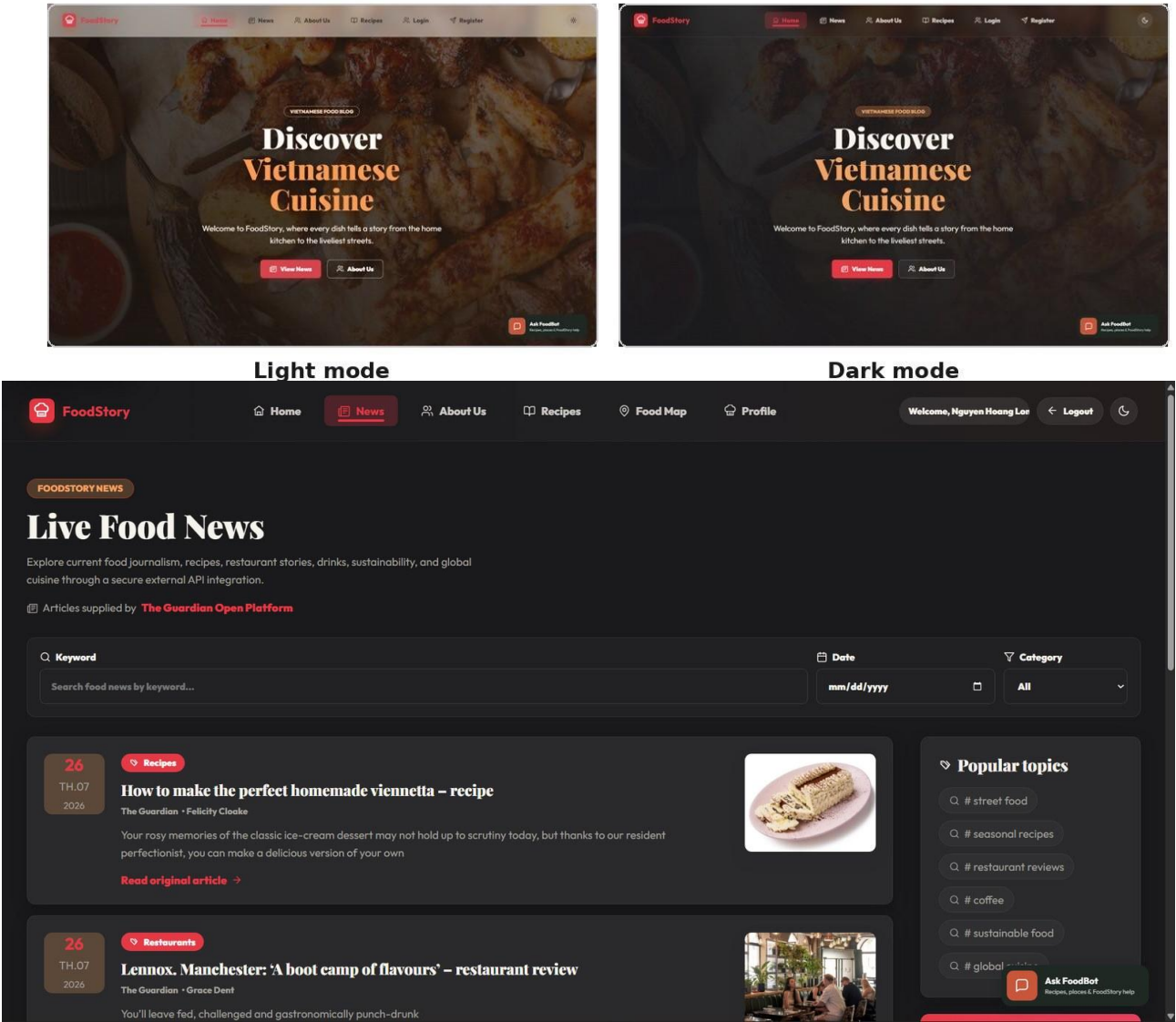


Figure 3A. The final News route retrieves Guardian food journalism through the server-side proxy and exposes keyword, date and category filtering in the deployed interface.

```
const response = await api.get('/news/external', {
  signal: newsRequestController.signal,
  params: { page, pageSize, search, date, category }
})

// A request identifier and AbortController prevent stale results.
```

3.5 About Page: From Required Exercise to Food Personality

About was the most difficult Stage 1 page because the required first name, last name, greeting, two radio buttons and changing image could easily look like a lab exercise placed inside an unrelated website. I retained every required interaction but reframed the radio choice as a food-personality selection, including Street Food Hunter and Home Cooking Lover. The selected state changes the image, card emphasis and summary message.

The page also uses an application description, cuisine imagery and summary statistics so the form is not the entire experience. The image-selection interaction remains the Stage 1 element I am most proud of because it demonstrates that assessment requirements can be integrated into the domain rather than merely displayed.

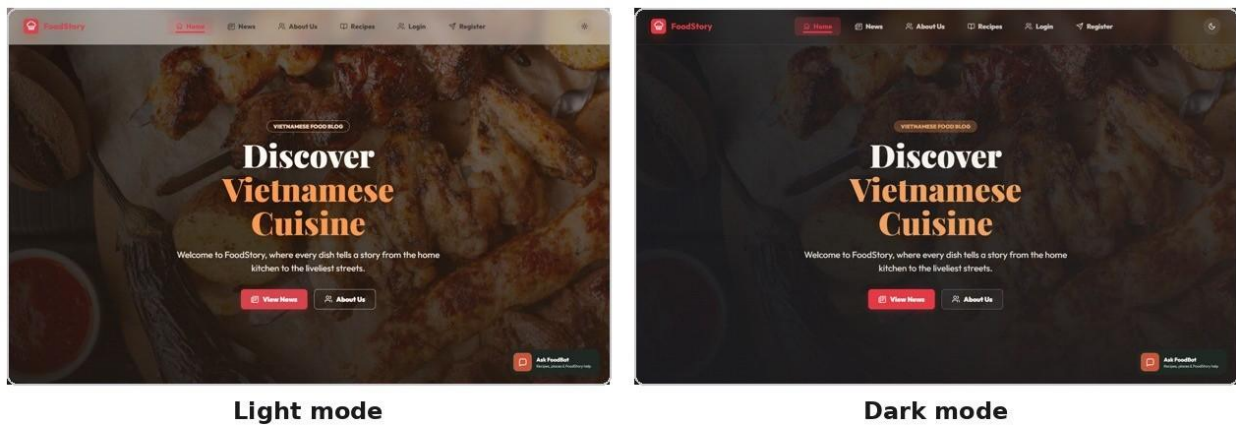


Figure 4. The required About form and radio-image behaviour were integrated into an enhanced Food Mood experience.

3.6 Responsive Design

Responsive behaviour was tested through Chrome DevTools and physical devices. The most persistent problem was the navigation bar. It began with three public routes, then accumulated Recipes, Food Map, Admin, Profile, account controls and a theme toggle. Each new route created spacing and overflow regressions. This taught me that a header should be designed for expected product growth rather than the current number of links.

The final mobile interface collapses the wider navigation into a phone-appropriate pattern. Bootstrap provides a mobile-first grid and responsive primitives, while custom CSS implements the distinctive typography, cards and theme tokens. Bootstrap's structure accelerated layout work, but the mixture of framework utilities and extensive custom rules increased maintenance cost.

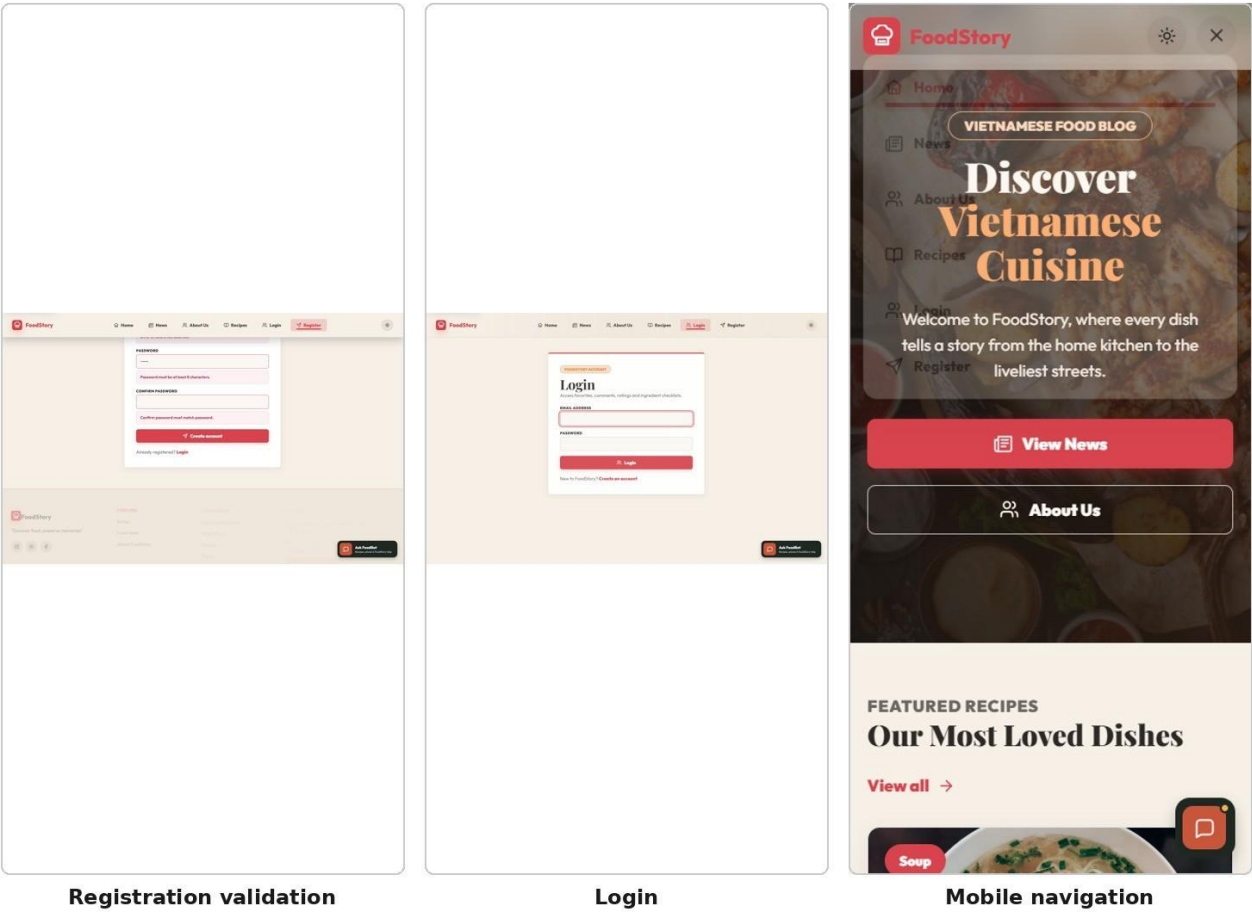


Figure 5. Registration validation, login and the mobile navigation demonstrate responsive and form behaviour.

3.7 Stage 1 Technology Comparison

Decision	Selected approach	Alternatives	Project-specific judgement
UI framework	Vue 3 Composition API	React; Angular	Vue was required and mapped visibly to directives, SFCs and computed state. React was personally familiar, while Angular would add unnecessary framework ceremony for the initial scope.
Build tool	Vite	Vue CLI; webpack	Vite provides a fast development server and optimised build workflow for modern projects (Vite, 2026). Vue CLI/webpack would provide mature configuration but more overhead.
Layout	Bootstrap grid + custom CSS	Tailwind; pure CSS	Bootstrap satisfied the explicit grid requirement; custom CSS delivered the

Decision	Selected approach	Alternatives	Project-specific judgement
			FoodStory identity. Tailwind could improve token consistency but would not automatically solve the design.
News data	Local JSON, later backend API	Hard-coded template; direct browser API	JSON cleanly separated data from markup for Stage 1. The final backend proxy protects credentials and standardises provider failures.
State style	Composition API	Options API	Options API may have reduced initial learning cost; Composition API created a direct path to reusable composables and larger feature organisation.

Table 3. Stage 1 technology comparisons.

3.8 Stage 1 Evaluation and Reflection

Requirement	Evidence	Status
Vite, Vue 3 and Router	Project manifests, routes and passing build	Implemented
Home page content	Hero, copy, images and current screenshots	Implemented
Local JSON News	12 historical records with four-per-page pagination	Implemented
Search by date/title/content/category	Historical data model and current UI contract	Implemented
About name greeting	Reactive v-model/computed interaction	Implemented
Two radio choices and changing image	Food Mood interaction	Implemented
Three device layouts	Desktop screenshots, phone screenshot and physical testing	Implemented
Formal Stage 1 user study	No tutor feedback or structured participant study	Not formally evaluated

Table 4. Stage 1 requirement evaluation.

My main Stage 1 dissatisfaction remains the navigation. My strongest learning was that UI polish is not separate from technical implementation: the route structure, responsive breakpoints and interaction model determine whether visual decisions survive later expansion. No specific tutor feedback was provided for this stage, so the evaluation relies on rubric mapping, source evidence and my own critical review.

4. Stage 2 - Evolving into a Full-Stack Interactive Platform

4.1 Architectural Transformation

Stage 2 changed FoodStory from a browser-based content prototype into a persistent multi-user application. Vue remained responsible for presentation and interaction, but shared state moved into Pinia, data came from Express APIs, ownership and role checks moved to server middleware, and MySQL became the authoritative data store. This separation was necessary because hiding a button on the frontend cannot protect a database mutation.

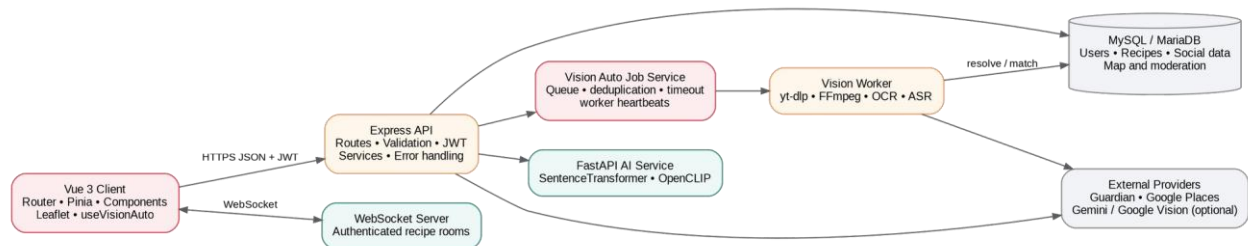


Figure 6. Current full-stack architecture and optional provider boundaries.

4.2 Pinia and Frontend State Boundaries

The final frontend contains domain stores for authentication, recipes, comments, ratings, favourites, checklists, administration, food spots, restaurants and UI notifications. Pinia is designed to share state across components and pages while providing testing and development tooling (Pinia, 2026). FoodStory benefits because the same recipe can appear in the catalogue, detail page and favourites, while a WebSocket rating event must update all relevant cached representations.

Not every value belongs in a store. Temporary input text, modal visibility and page-local loading state remain inside components or composables. This boundary reduces global coupling. Cross-store updates are still a maintainability risk; the current design works, but a larger application would benefit from stronger typed event contracts or a dedicated cache/query layer.

4.3 Authentication and Roles

Registration validates username, email and password limits, performs a duplicate query, hashes the password using bcrypt and assigns the user role. Login retrieves the account through a parameterised query, rejects banned accounts, compares the password hash and signs a JWT. The token is then used for protected HTTP requests and authenticated WebSocket subscriptions. Backend middleware remains the security authority even when the router and v-permission directive hide unavailable UI actions.

Role	Accessible experience	Restricted actions
Guest	Home, current News, About, recipe discovery/detail, FoodBot public guidance and limited Food Map preview.	Persistent social actions, profile, submission and administration.
Authenticated user	Comments, ratings, favourites,	Administrative governance and other

Role	Accessible experience	Restricted actions
	checklists, profile, submissions, personal map functions and authenticated FoodBot context.	users' owned mutations.
Administrator	All user capabilities plus dashboard, content CRUD, moderation, user-role/ban controls and comment governance.	Still constrained by backend validation and audit rules.

Table 5. Stage 2 role model.

4.4 Recipe Ecosystem

The recipe feature became the centre of Stage 2. A recipe is no longer a static card; it contains category, tags, description, image, ingredients, instructions, preparation and cooking time, servings, difficulty, nutrition, moderation state and social aggregates. Users can move from discovery to cooking preparation through search, filters, pagination, favourites and generated shopping checklists.

This is the Stage 2 result I am most proud of. The value does not come only from the number of seeded recipes, but from the complete workflow around each recipe. FoodStory therefore differs from a read-only catalogue because a recipe becomes a persistent shared object with user activity and ownership.



Figure 7. Recipe discovery and the detailed interaction surface form the Stage 2 core.

Final UX Refinement from Informal Usability Feedback

A final informal formative review with one user identified interaction friction rather than missing core functionality. During recipe search, unrelated discovery collections remained visible and weakened the meaning of the query. Recipe Detail exposed every section as one long document, while large comment lists pushed primary cooking content further down the page. The user also reported that excessive transition movement could feel uncomfortable.

The final implementation responds with an explicit focused-search mode that hides unrelated collections and trending suggestions while showing the active query, result count and clear-search action. Recipe Detail now provides a sticky desktop Recipe guide with active-section highlighting, direct navigation to Overview, Ingredients, Instructions, Nutrition, Ratings, Comments and More recipes, and collapsible high-density sections. Comments load in bounded groups of five and display the remaining count instead of rendering the entire discussion immediately. Motion was reduced and the interface retains support for reduced-motion preferences.

These changes are progressive disclosure rather than content removal: the full ingredients, instructions, nutrition and discussion remain available, but the initial information density and scrolling burden are lower. The evaluation is intentionally described as formative because one participant and no formal task protocol cannot support a general usability claim.

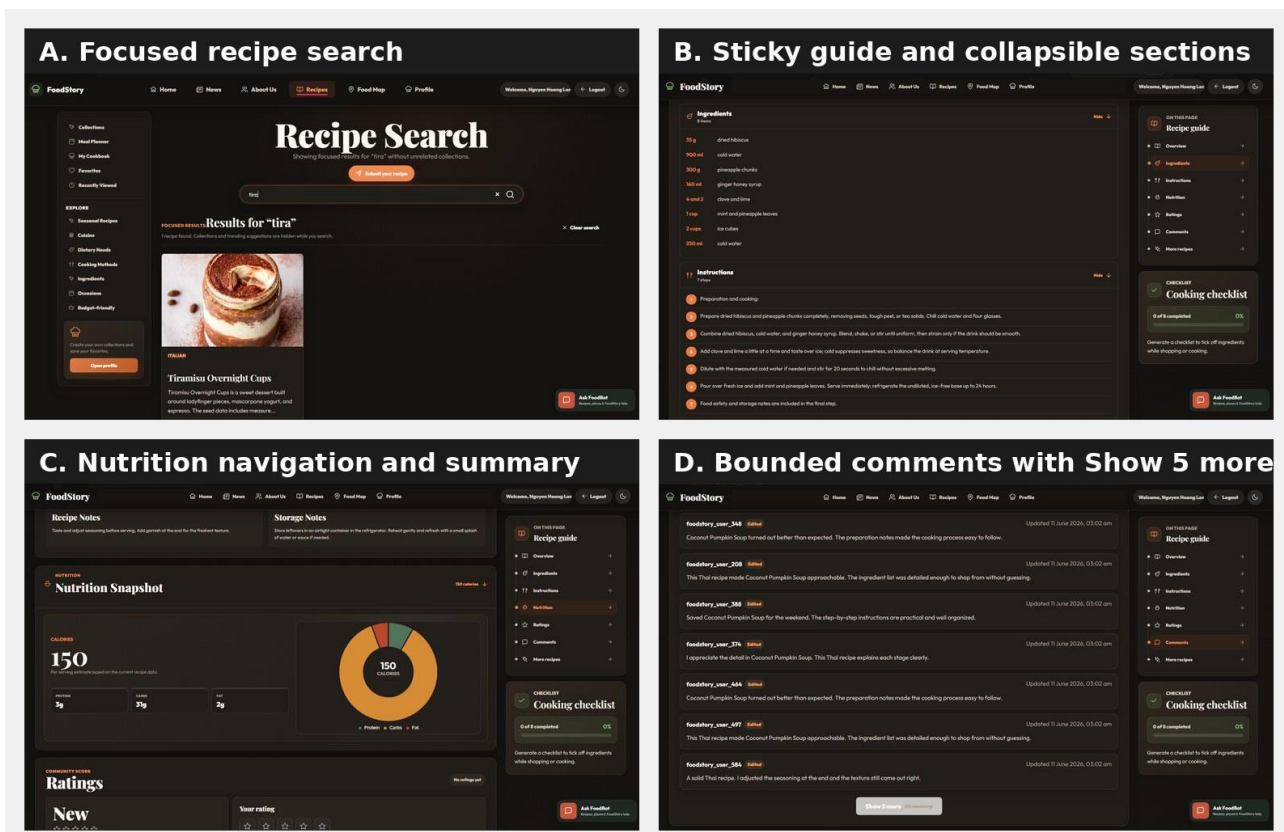


Figure 7A. Final recipe UX refinement. (A) Focused search hides unrelated collections. (B) A sticky guide and collapsible sections reduce navigation effort. (C) Nutrition remains directly accessible. (D) Comments load five at a time with a remaining-count indicator.

4.5 Comments, Ratings and Favourites: The Hardest Stage 2 Problem

Comments, ratings and favourites were the hardest Stage 2 features because correct behaviour had to extend beyond the browser that initiated the action. A successful local update is insufficient when another user is viewing the same recipe. The implementation had to preserve database consistency, ownership, exact-once UI reconciliation and low-latency delivery.

The rating route uses a transaction, locks the approved recipe, inserts or updates one user-recipe row, recalculates the aggregate, commits, and only then broadcasts the canonical result. A unique database constraint on (user_id, recipe_id) prevents concurrent duplicate ratings. Comment edit and delete

routes query the stored owner and return 403 when the authenticated user does not own the comment.

Code listing 2. Rating persistence and broadcast order.

```
await connection.beginTransaction()
await connection.execute(
  `INSERT INTO ratings (user_id, recipe_id, rating_value)
  VALUES (?, ?, ?)
  ON DUPLICATE KEY UPDATE rating_value = VALUES(rating_value)`,
  [req.user.id, recipeId, ratingValue],
)
const [summary] = await connection.execute(
  `SELECT AVG(rating_value) AS average_rating, COUNT(*) AS total_ratings
  FROM ratings WHERE recipe_id = ?`, [recipeId])
await connection.commit()
broadcastToRecipe(recipeId, { type: 'rating_updated', ...summary[0] })
```

4.6 WebSocket Real-Time Synchronisation

WebSocket was suggested by the tutor and became the appropriate solution for event-driven recipe updates. The server authenticates the first subscription message with the JWT, places the socket into a recipe-specific room, monitors liveness with ping/pong and removes failed connections. REST remains responsible for writes; WebSocket distributes a committed event. This division avoids using a transient socket message as the data source of truth.

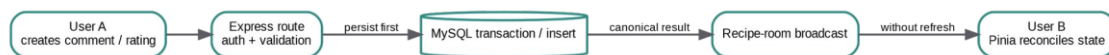


Figure 8. Real-time recipe interaction persists before broadcasting to subscribed clients.

The frontend updates Pinia stores rather than manipulating the DOM directly. It de-duplicates comments by stable database ID because the initiating user may receive both the REST response and the WebSocket event. An earlier two-browser sequence showed User B receiving User A's comment and updated rating without refreshing, and an earlier 4/4 acceptance script covered create, edit, delete and rating broadcasts. These checks remain historical integrated evidence and were not rerun after the final UX/provider refinement.

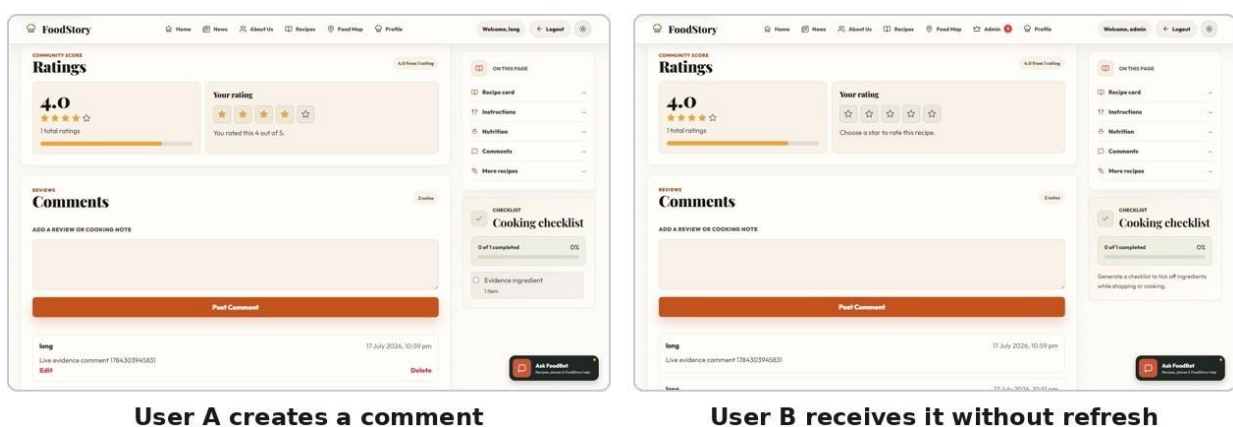


Figure 9. User A creates a comment and User B receives it through the recipe WebSocket room without refreshing.

The current room map is process-local. This is sufficient for the academic single-process deployment but not horizontally scalable. A production multi-instance architecture would require a shared broker such as Redis pub/sub. The browser WebSocket API also does not provide automatic backpressure, so high-volume deployments would require queue limits or event coalescing (MDN, 2026).

4.7 Persistence and Relational Integrity

MySQL was selected because FoodStory contains explicit relationships and uniqueness rules: users submit recipes, administrators moderate them, recipes have many ingredients and tags, users create comments and one rating/favourite/checklist relationship per recipe. These constraints are more safely enforced in the database than through a sequence of client checks.

The schema uses foreign keys, composite keys, check constraints, indexes and cascade behaviour. For example, favourites use (user_id, recipe_id) as the primary key; ratings enforce a 1-5 range and a unique user-recipe pair; checklist items cascade from their parent; and submitted_by or moderated_by can be set to null without deleting a historical recipe. Parameterised mysql2 queries protect query structure from user input, consistent with OWASP SQL-injection guidance (OWASP, 2021a).

4.8 User Contribution and Administration

Authenticated users can submit recipes, which enter a pending moderation state rather than immediately becoming public. Administrators can approve, reject and manage content through protected routes. This workflow turns contribution into governed community content and creates clear accountability for moderation decisions.

The Admin Dashboard is functionally complete for individual governance operations but remains unvalidated at production scale. It combines overview metrics with searchable and paginated recipe management, pending review, required rejection reasons, user search and role filtering, role updates, ban/unban controls, comment search and comment deletion. The limitation is operational efficiency rather than the absence of governance: a larger system would benefit from audit timelines, queue priority and bulk actions.

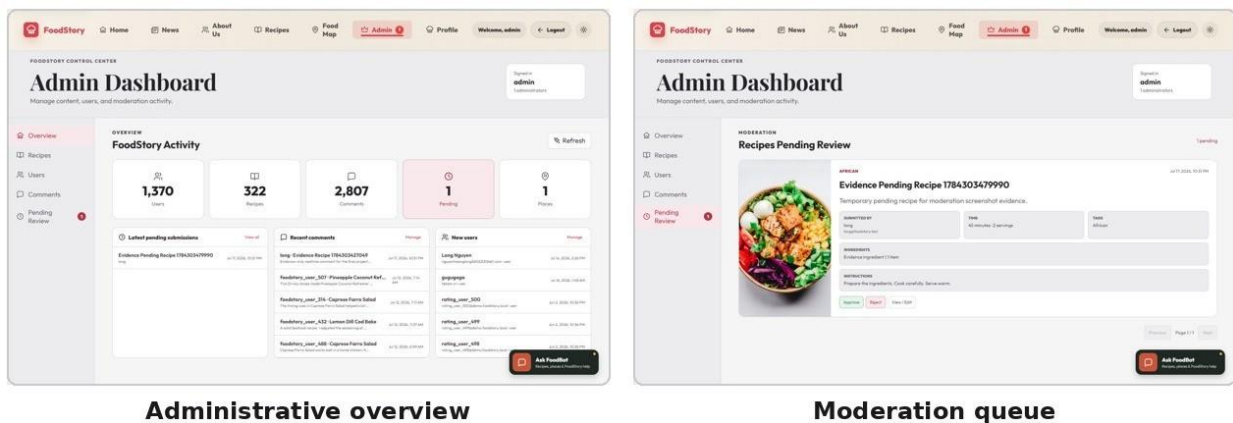


Figure 10. The administrator dashboard and moderation queue provide role-specific governance.

4.8.1 Operational Governance and Server-Side Authority

Vue centralises Admin page state in a domain store, but privileged authority remains on the server. Every Admin API requires a verified JWT and administrator role, so the router redirect is a usability guard rather than the only security control. The final evidence run reproduced the dashboard and a non-admin request to /api/admin/stats returned HTTP 403. Destructive role, ban and comment actions were not repeated against shared seeded accounts; their controls are source- and test-verified. This separation between visible interface affordances and backend authorisation prevents a modified client from becoming an administrator.

4.8.2 Submission-to-Publication Workflow

Community recipe contribution is implemented as a moderated state transition rather than an immediate public insert. A signed-in non-admin submits a validated recipe with pending status and submitter identity. Public catalogue/detail queries exclude it, while the submitter and administrator can retrieve it through role-sensitive paths. Approval records moderation identity and makes the same entity publicly visible; rejection requires a reason. The final browser run reproduced submission, queue presence, approval, public visibility and cleanup. This cross-role lifecycle demonstrates persistence, policy and accountability rather than four unrelated screens.

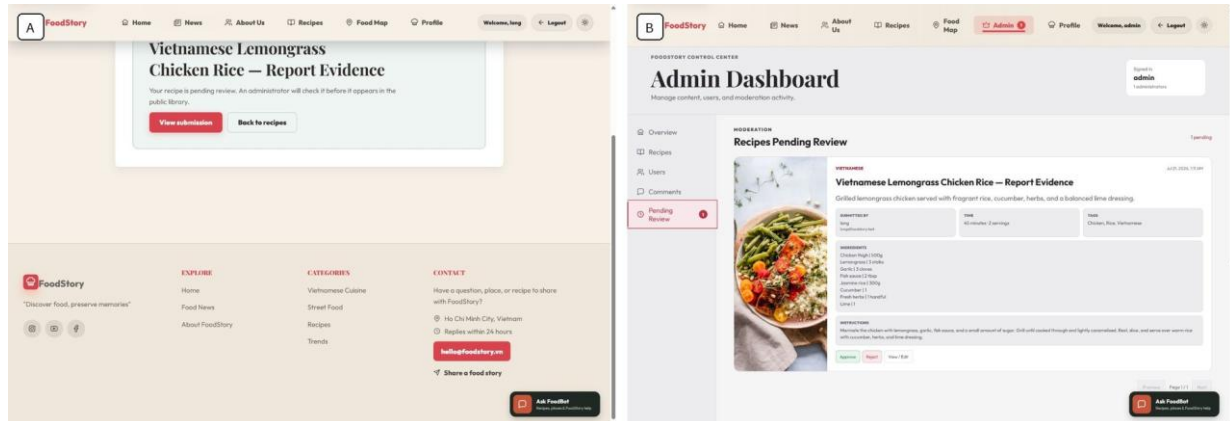


Figure 10A. A normal-user recipe submission remains pending until the same record appears in the administrator review queue; the temporary evidence record was deleted after capture.

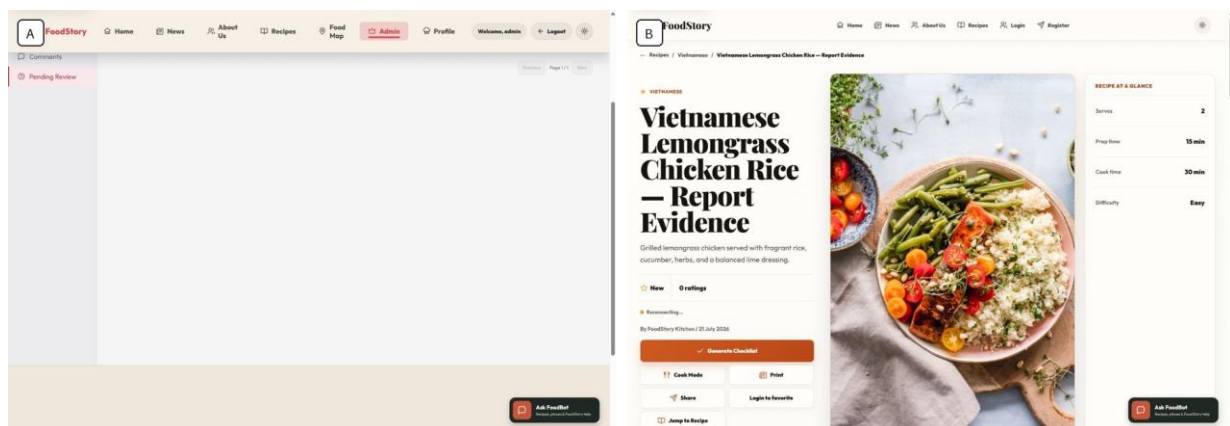


Figure 10B. Approval clears the pending queue and makes the same recipe readable through the public recipe route.

4.9 FoodBot as a Complementary Full-Stack Extension

FoodBot is documented as a complementary full-stack extension rather than the assessed Stage 3 advanced technique. It is implemented as a routed evidence system, not one unrestricted language-model prompt. Queries are classified into structured FoodStory data, website guidance, private account data, optional external research or general culinary guidance. Recipe ingredients, quantities, nutrition and cooking-time questions are answered from stored application data and expose FoodStory source cards. Account-specific favourites, checklists and saved places require an authenticated user identity. This local-data-first design makes provenance visible and preserves useful behaviour when an external provider is unavailable.

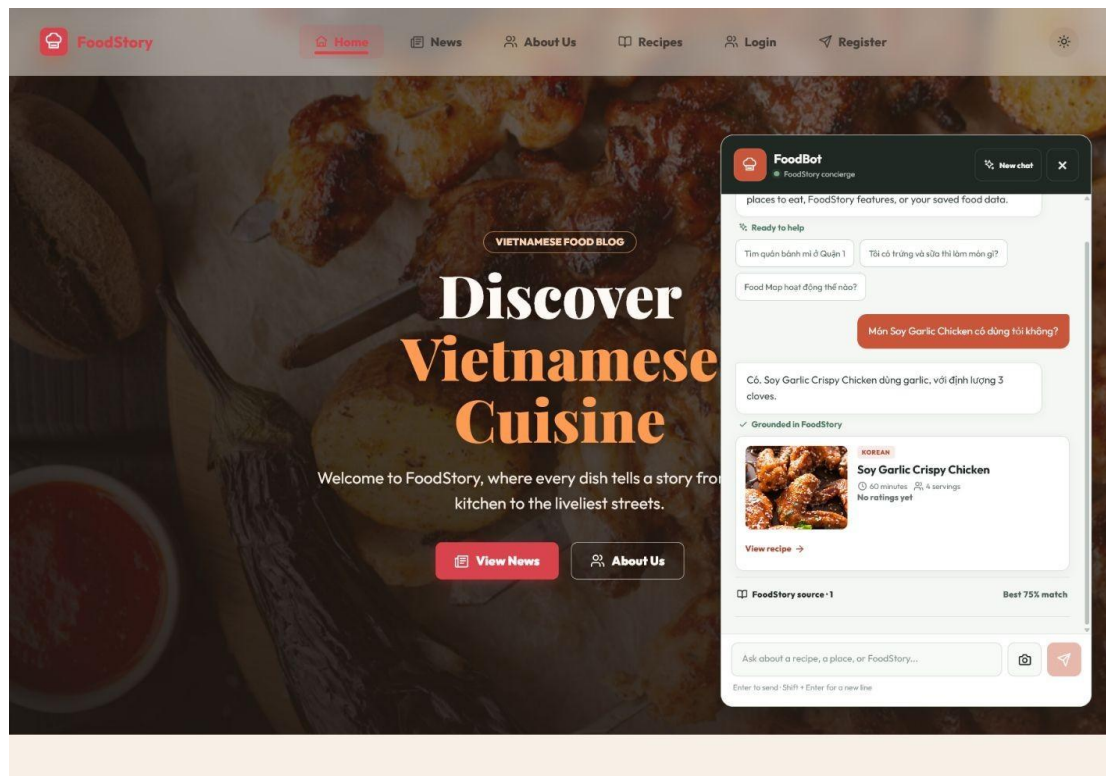


Figure 10C. FoodBot answers an ingredient question from the stored Soy Garlic Crispy Chicken recipe and exposes the supporting FoodStory source card.

4.9.1 Bounded Conversation Memory

Conversational continuity is bounded at both the browser and API boundary. History is partitioned between guest browser profiles and signed-in accounts, sanitised before reuse and capped by the chatbot route. The backend derives a small context model, including recent recipe or restaurant identifiers, rather than treating the full transcript as trusted system instructions. This supports follow-up questions while limiting storage growth, prompt cost and instruction-confusion risk. The trade-off is that history is device-local and is not synchronised across browsers; starting a new conversation explicitly clears the active context.

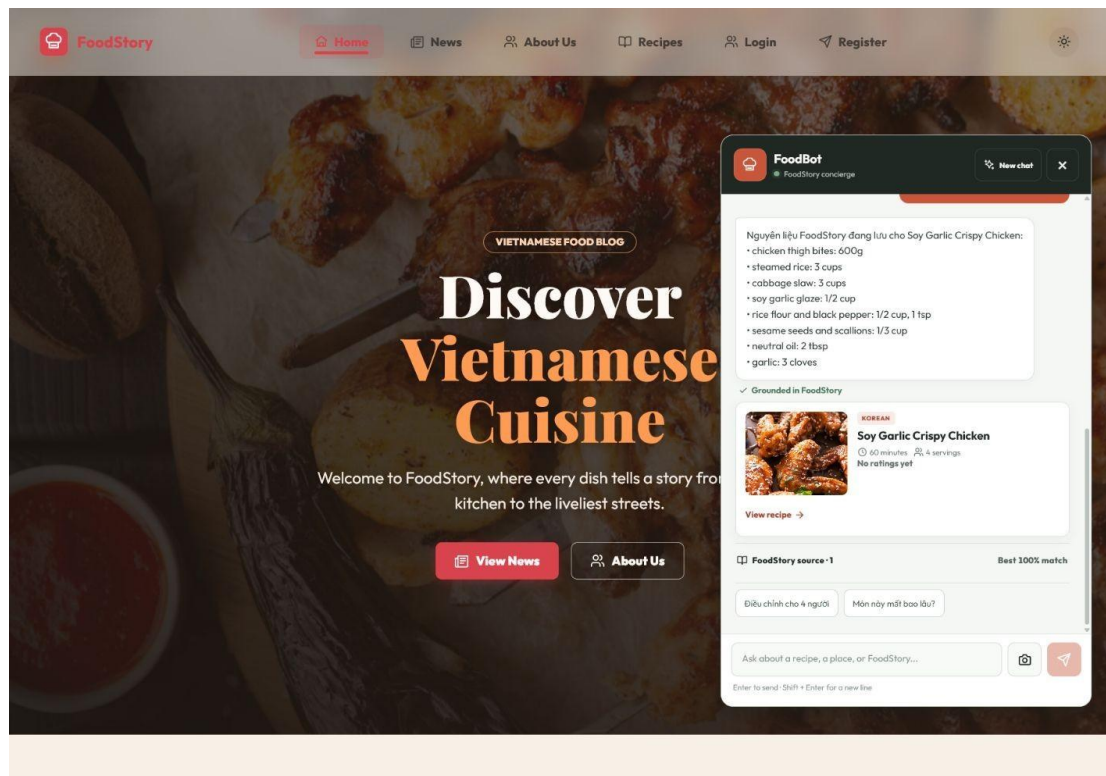


Figure 10D. The Vietnamese follow-up resolves the immediately preceding dish through bounded recent context.

4.9.2 Image-Based FoodStory Search Boundary

FoodBot also exposes image-based discovery through an isolated AI-service boundary. The browser accepts a bounded image, Express validates multipart type and size, and an authenticated server-to-server request obtains an embedding from FastAPI before comparing it with stored FoodStory records. The final source propagates a bearer token, fails closed in production, rate-limits the AI boundary, validates image decoding and rechecks connected peers for remote images. The 6/6 deterministic smoke suite covers these controls. The final browser-automation run encountered a genuine MIME validation error during upload, so successful visual inference remains a source- and security-verified capability rather than a live accuracy result.

4.10 Stage 2 Technology Decisions

Decision	Selected	Alternative	Evaluation
Shared state	Pinia	Component-only state; Vuex	Domain stores support cross-page and WebSocket updates with less ceremony than Vuex, but require disciplined boundaries.
API client	Axios	Native Fetch	Central token, timeout, error and cancellation behaviour justified Axios for many authenticated requests.
Backend	Express	NestJS; FastAPI	Express matched the existing JavaScript route/service design. NestJS

Decision	Selected	Alternative	Evaluation
			could improve large-team structure; FastAPI was reserved for Python ML.
Persistence	MySQL/MariaDB	MongoDB; SQLite	Relational constraints directly model ownership, moderation and unique social relationships.
Authentication	JWT + bcrypt	Server sessions; OAuth	JWT unified HTTP and WebSocket identity. Production should reassess localStorage and revocation.
Realtime	WebSocket	Polling; SSE; Socket.IO	WebSocket supports bidirectional low-latency room events; polling is simpler but wasteful when idle.
API style	REST	GraphQL	Resource routes made CRUD, ownership and tests easy to trace; GraphQL would add resolver and authorisation complexity.
Charts	Chart.js	D3	Chart.js was proportionate for bounded nutrition/admin metrics; D3 would add control at higher complexity.
Conversational assistance	Routed local-data-first FoodBot with bounded context	Single unrestricted LLM prompt; static FAQ	Preserves provenance and provider fallback, but increases routing, memory and trust-boundary complexity.

Table 6. Stage 2 technology decisions.

4.11 Stage 2 Evaluation

Requirement	Evidence	Status
Components, router, directives	Modular views/components, metadata guards, v-focus and v-permission	Implemented
Forms and validation	Registration, recipe form, comments and backend validation	Implemented
Mobile-first responsiveness	Desktop and phone evidence; custom breakpoints and Bootstrap grid	Implemented
Accessibility	Semantic labels, native controls, focus states and live regions	Implemented with limitations

Status		
Registration/login and differentiated visibility	JWT, router guards and middleware; earlier 87/87 focused API run retained as historical evidence.	Implemented
Search/filter/pagination	Recipes and current News API	Implemented
Social features	Ratings, comments, favourites and real-time updates	Implemented
Authorised CRUD	Owner comment CRUD, submission and admin management	Implemented
Persistent data	MySQL schema and migrations; earlier 5/5 audit retained as historical evidence.	Implemented
Real-user UX validation	One informal formative review directly informed focused search, progressive disclosure, bounded comments, reduced motion and address-assisted map placement.	Formative evidence only
Grounded FoodBot assistance	Runtime grounded and follow-up evidence; routing/memory/security tests; visual-search path source verified	Implemented with limitations

Table 7. Stage 2 requirement evaluation.

No tutor feedback was provided after Stage 2. I therefore evaluated it through direct browser testing, API scripts, source review and database behaviour. AI assistance accelerated implementation, but I kept code only after reading it, executing the relevant flow and rerunning tests. This verification process is particularly important for real-time features, where a visually plausible component can still be inconsistent across users.

5. Stage 3 - Advanced Vue Composable and Vision-Assisted Food Discovery

5.1 Advanced Technique Definition and Assessment Relevance

Assessed advanced technique

A reusable Vue 3 composable that manages long-running, cancellable Vision Auto jobs and safely synchronises structured food-discovery outcomes with an interactive Leaflet Food Map.

Assessment boundary

The assessed core is useVisionAuto and its Vue/Leaflet integration. Express job coordination, FastAPI, OCR/ASR, provider adapters, SSRF controls and FoodBot are supporting infrastructure or complementary features; they are discussed only where they shape the interface contract, lifecycle, security or evidence.

The assessed contribution is the Vue-side orchestration of a multi-stage asynchronous system whose outcome may be successful, reviewable, unavailable, cancelled or unresolved. useVisionAuto coordinates reactive state, computed eligibility and progress, polling, abort signals, backend cancellation, stale-response protection, retry/reset behaviour and component-lifecycle cleanup. A user can submit a video URL, observe bounded progress, cancel without a superseded response reopening the panel, retry under a new run identity and receive a safely adapted result. The backend and AI pipeline support this interaction; they are not used to imply that recognition accuracy has been established. The assessed Vue technique is the composable and its integration with FoodMapView.

The 20 July revision strengthened this boundary after focused tests exposed two genuine lifecycle risks. A late create-job response could contaminate the active job ID of a newer run, and an already scheduled polling delay survived cancellation. The corrected implementation captures job IDs per run, cancels stale late-created jobs and makes the polling delay abort-aware. These are advanced Vue concerns because they control which asynchronous continuation is permitted to mutate current reactive state (FoodStory, 2026d).

5.2 Food Map as the Existing Product Context

The Food Map existed before Vision Auto because I wanted FoodStory to preserve personal food memories rather than remain only a shared recipe database. Users can save places they visited, dishes they tried, ratings, notes and tags. Leaflet provides map and marker interaction, while MarkerCluster reduces overlap in dense urban areas.

Stage 3 did not replace this map. FoodMapView remains the imperative owner of the Leaflet instance, marker clusters, stores, filters, coordinate picking, focus and teardown. The composable supplies bounded reactive job state. This separation preserved product behaviour and avoided mixing network timers and result adaptation with marker creation. The final main view contains 3,124 lines compared with 8,481 in the earlier monolith, while the complete distributed feature contains 9,091 lines. The split reduced the responsibility concentrated in one view; I did not measure a corresponding frame-rate improvement.

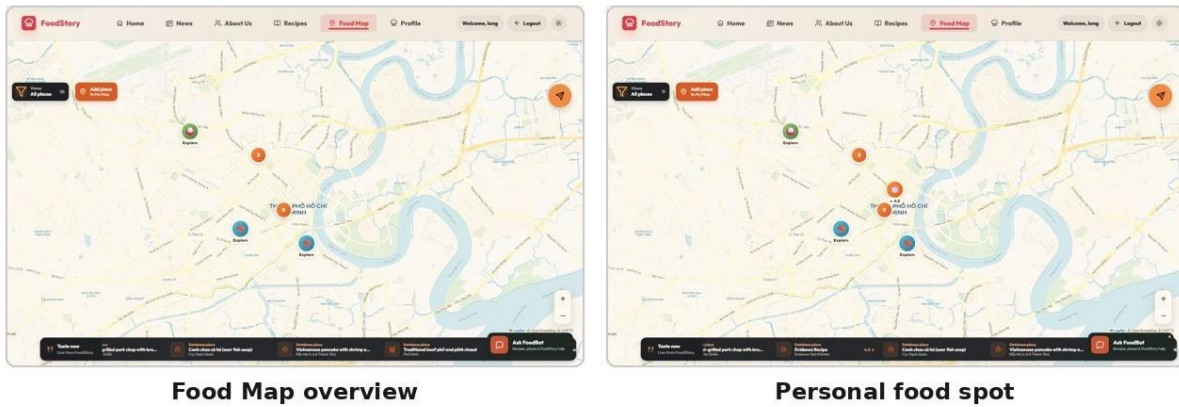


Figure 11. The Food Map combines shared restaurant records with user-owned food memories.

5.2.1 Progressive Guest-to-Authenticated Access

Food Map applies progressive access rather than a login wall. Guests can explore verified restaurants and community places through a labelled preview, while the banner explains that saving and managing a personal journey requires authentication. A signed-in user gains My Map, Statistics, Add Place and Vision Auto. Personal create, edit and delete queries are scoped by the authenticated user identifier. The final browser evidence therefore demonstrates a deliberate product rule: authentication unlocks persistence and personalisation rather than blocking initial discovery.

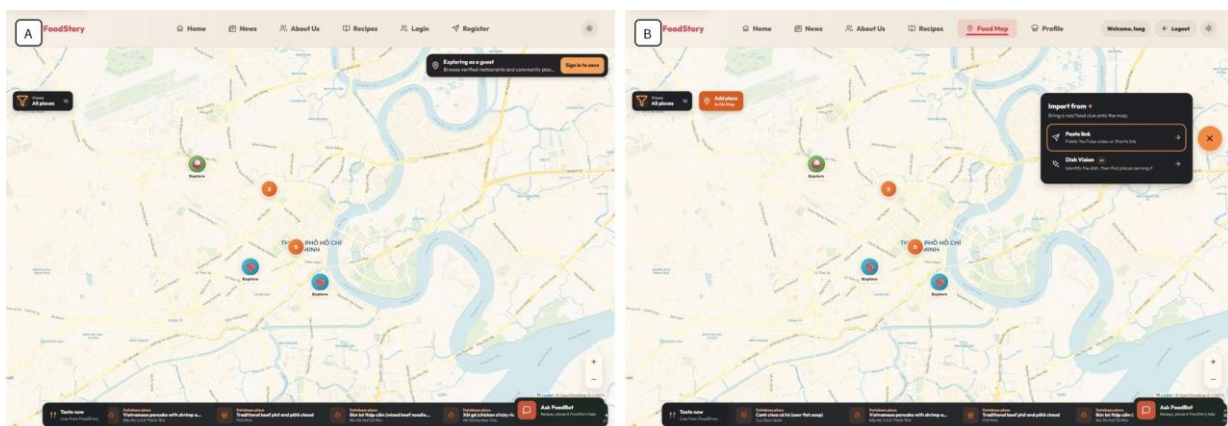


Figure 11A. Guests can explore community places, while authenticated users gain personal persistence, contribution and source-import entry points.

5.2.2 Intent-Oriented Discovery Modes

Map exploration is organised by intent rather than one undifferentiated marker list. Near map prioritises distance when location is available; Top rated emphasises quality; Budget picks applies price constraints; and Surprise me reduces choice effort. Search, district/category/rating filters, layers and marker clustering remain orthogonal controls. The map and result tray share the filtered discovery set, so selecting a result can focus its marker and marker interaction can expose the same detail model. Browser geolocation remains permission-dependent, and clustering has not been load-tested at production scale.

Address-Assisted Place Creation

Informal usability feedback identified manual map pinning as unnecessarily slow when a user already knew the address. The final Add Place workflow therefore prioritises address or place-name search while preserving manual map selection as a fallback and fine-adjustment tool. The user enters a bounded query, reviews provider candidates, selects the closest result, and the map navigates to that position before the place is saved.

The interface identifies Geoapify as the precise provider and OpenStreetMap as the fallback data source. When a house number cannot be verified, FoodStory presents nearby or street-level candidates and explicitly asks the user to choose the closest result and adjust the pin. This avoids silently treating approximate geocoding as an exact private location.

The same final map surface supports the dish-first Vision flow: users confirm a bounded visual dish candidate, search around the current map area, inspect provider-labelled alternatives, view a result on the map or add it to FoodStory. A place-name or category match remains a discovery suggestion and does not prove that the venue is the original filming location or currently serves the dish.

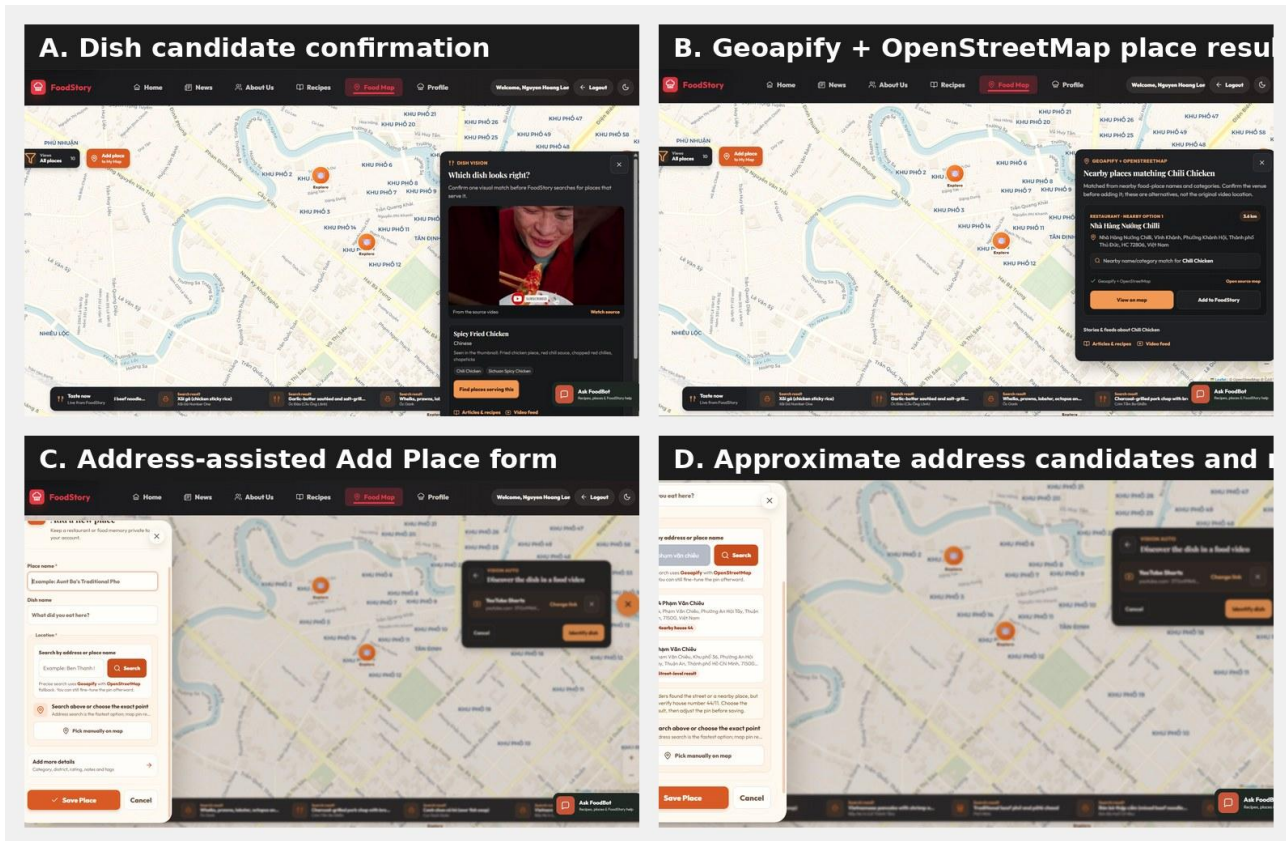


Figure 11B. Final Food Map refinement. (A) The user confirms a visual dish candidate. (B) Geoapify and OpenStreetMap provide a labelled nearby alternative. (C) Add Place supports address search. (D) Approximate results preserve manual pin adjustment and explicit user confirmation.

5.3 Evolution from Image OCR to Vision-Assisted Video Discovery

The implementation evolved through five bounded stages: single-image OCR, early video sampling, explicit Track 1/Track 2 routing, candidate-first review and dish-first discovery. The first prototype

worked only when a user had already selected a frame containing a sign or address. Moving to public short-video URLs changed the problem from text recognition to evidence selection: subtitles, prices, phone numbers, animated overlays, reposted footage and multiple locations all compete with the useful clue.

Phase	Primary idea	Key learning
A. Single-image OCR	OCR a selected sign/address frame.	Strong evidence is usable when the correct frame is already known.
B. Early video sampling	Prioritise beginning/end frames.	Automatic frame selection is an independent recall and latency problem.
C. Track 1 / Track 2	Separate explicit metadata from weak visual evidence.	Stable strong-evidence routing should not be destabilised by experimental recovery logic.
D. Candidate-first	Preserve bounded partial clues as review evidence.	Plausible text must not automatically become a map location.
E. Dish-first	Identify a dish, then search serving-place alternatives.	A useful reviewable result is preferable to an unsupported filming-location claim.

Table 8. Vision Auto evolution timeline.

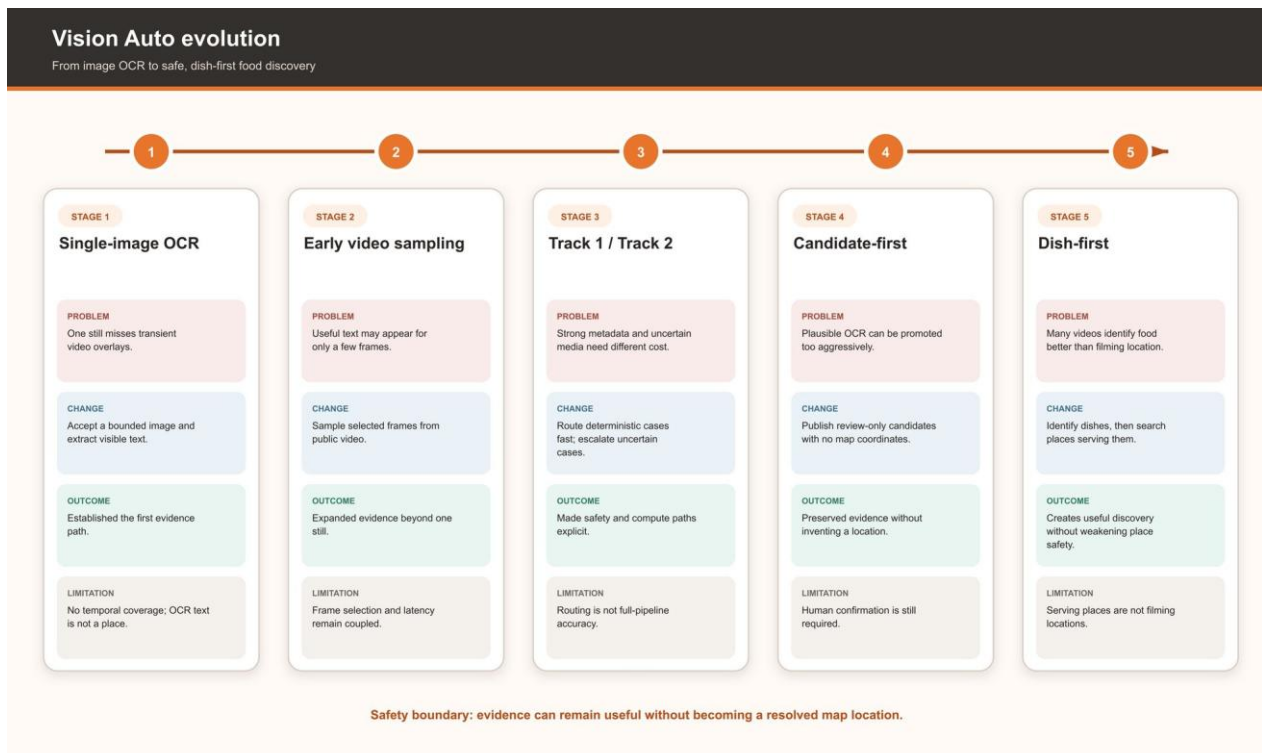


Figure 12. Evolution from single-image OCR to dish-first discovery, including the problem, engineering change, outcome and remaining limitation at each stage.

5.4 The Accuracy-Frame-Latency Problem

Video discovery creates a three-way tension. Sampling more frames may expose a short-lived address, but increases media acquisition, crop generation and OCR work. Aggressive promotion improves apparent match rate but increases false locations. Conservative routing protects users but produces more review or not-found outcomes. Repeated experimentation showed that no single OCR parameter solved all three constraints. The final boundary therefore prioritises safety: only resolver-backed

FoodStory or external place records can become map targets; OCR or ASR evidence remains review-only until confirmed.

The compatible benchmark corpus contains deterministic router fixtures rather than common labelled media. Consequently, frame count, OCR and ASR latency, exact-place accuracy, dish accuracy and end-to-end user latency remain open evaluation requirements instead of estimated values.

5.5 Track 1, Track 2 and Candidate-First Evidence

Track 1 handles strong metadata/address cases through conservative deterministic routing. Track 2 V3 performs bounded evidence recovery for uncertain cases. Candidate-first processing prevents a plausible OCR string from becoming a real place card merely because it has a confidence score. Review candidates explicitly carry `reviewRequired = true` and `canAutoResolve = false`, contain no map coordinates and must not be described as resolved locations.

The same 30 labelled router cases were replayed five measured times for Track 2 V3 and Final. Both versions matched the intended fixture routing in 96.67% of cases, with 0.00% false promotion, a 70.00% review-route proportion and 0.00% timeout. These percentages describe agreement with a narrow deterministic fixture only; they are not media recognition accuracy.

5.6 Advanced Vue Design of useVisionAuto

5.6.1 Public Composable Contract

Vue defines a composable as a function that encapsulates reusable stateful logic and recommends explicit cleanup for side effects when the consumer unmounts (Vue.js, 2026a). `useVisionAuto` applies this model to a long-running job rather than to a simple data fetch. Five network functions are injectable - create job, get job, cancel job, discover dish and search dish places - so tests can substitute deterministic implementations without a live backend or provider.

Contract group	Members	Responsibility
Input	<code>inputMode</code> , <code>url</code> , <code>inputError</code> , <code>hasSubmittedSource</code>	Own URL-entry state and validation feedback.
Lifecycle	<code>state</code> , <code>elapsedSeconds</code> , <code>isAnalyzing</code> , <code>analyzingCopy</code>	Represent progress, cancellation and terminal outcomes.
Result	<code>result</code> , <code>errorMessage</code> , <code>sourceSummary</code>	Expose bounded result and sanitised public error state.
Eligibility	<code>hasValidUrl</code> , <code>canAnalyze</code>	Derive whether a new run can start.
Input actions	<code>openLink</code> , <code>backToMenu</code> , <code>setUrl</code> , <code>clearUrl</code>	Manage the compact import surface.
Job actions	<code>submit</code> , <code>cancel</code> , <code>retry</code> , <code>reset</code> , <code>dispose</code>	Coordinate run identity, timers, HTTP and backend cancellation.
Dish-first actions	<code>submitDishDiscovery</code> , <code>selectDish</code>	Identify a dish and search serving-place alternatives.
Result action	<code>clearResult</code>	Dismiss the current outcome without changing map ownership.

Table 8A. Public contract of `useVisionAuto`.

5.6.2 Reactive State and Computed Derivations

Mutable public state is held in refs, while computed values derive busy status, URL validity, submission eligibility, source summary and progress copy. Network side effects begin from explicit

actions rather than watchers, which makes the cause of each transition visible in tests. The authoritative state ref prevents multiple redundant booleans from disagreeing about whether a job is queued, analysing, resolved or cancelled.

5.6.3 Polling and Backend Job Coordination

submit validates the URL, invalidates any previous operation, allocates a new AbortController and monotonically increasing run ID, starts elapsed timing, creates a backend job and polls every 1.5 seconds until a terminal status. Backend implementation stages are deliberately reduced to fast_analysis, deep_analysis and resolving before reaching the panel. This keeps progress meaningful without exposing worker internals.

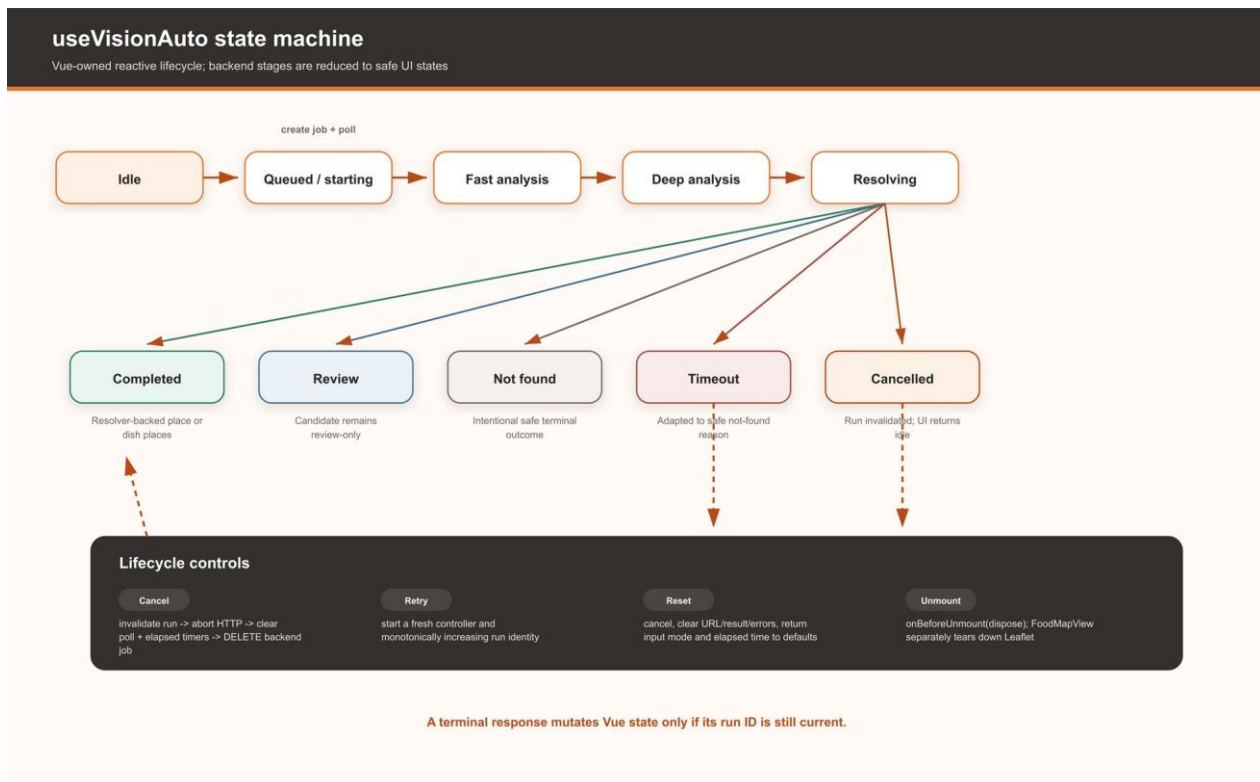


Figure 13. Reactive lifecycle and terminal uncertainty states managed by useVisionAuto; backend stages are adapted before reaching the UI.

5.6.4 Cancellation and Lifecycle Cleanup

Cancellation is multi-layered. The run guard is invalidated first, so a late continuation cannot mutate refs. The HTTP controller is aborted, the current polling delay and elapsed interval are cleared, and a best-effort DELETE is sent for the active backend job. onBeforeUnmount(dispose) applies the same cleanup when the component leaves the page. FoodMapView separately removes its own listeners, timers, animation frames, marker collections and Leaflet map, preserving distinct lifecycles.

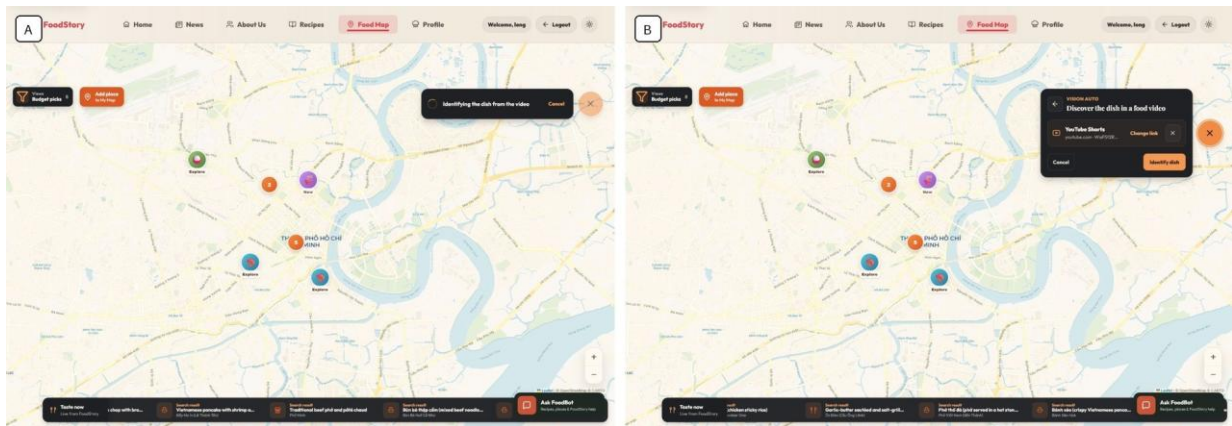


Figure 13A. Vision Auto exposes cancellation while analysis is active and returns to a reusable input state after aborting the current run; the UI does not claim a separate persistent Cancelled result.

5.6.5 State-Run and Race-Condition Protection

Every operation receives a new run identity, and each asynchronous continuation checks that identity before polling or assigning a result. Focused tests exposed a subtle defect: a superseded create-job response could overwrite the shared activeJobId after a newer run had begun. The revision captures the backend job ID as run-local state and cancels a stale late-created job instead of adopting it. A second test exposed a scheduled polling timeout that remained queued after cancellation; the delay now resolves immediately on abort.

Code listing 3. Simplified run-local job identity and abort-aware polling boundary.

```
const requestId = runGuard.start()
const controller = new AbortController()
const created = await createJob({ url: url.value.trim(), signal: controller.signal })
const runJobId = created.data.jobId

if (!runGuard.isCurrent(requestId) || controller.signal.aborted) {
  await cancelJob(runJobId)
  return null
}

while (!terminalStates.includes(job.status)) {
  await waitForNextPoll(controller.signal)
  if (!runGuard.isCurrent(requestId)) return null
  job = (await getJob(runJobId, { signal: controller.signal })).data
}
```

5.6.6 Result Adaptation and Leaflet Synchronisation

The response adapter is a client safety backstop. A public place requires a supported source type, stable ID, name and address. An external place additionally requires finite coordinates and a provider place ID. Review evidence is accepted only when explicitly review-only and is never added to mapTargets. Malformed positive results degrade to safe not_found. FoodMapView then resolves a FoodStory ID against real store data, focuses an existing marker where possible and uses finite coordinates only for validated external places. Review evidence enters the existing manual place-review workflow rather than being silently persisted.

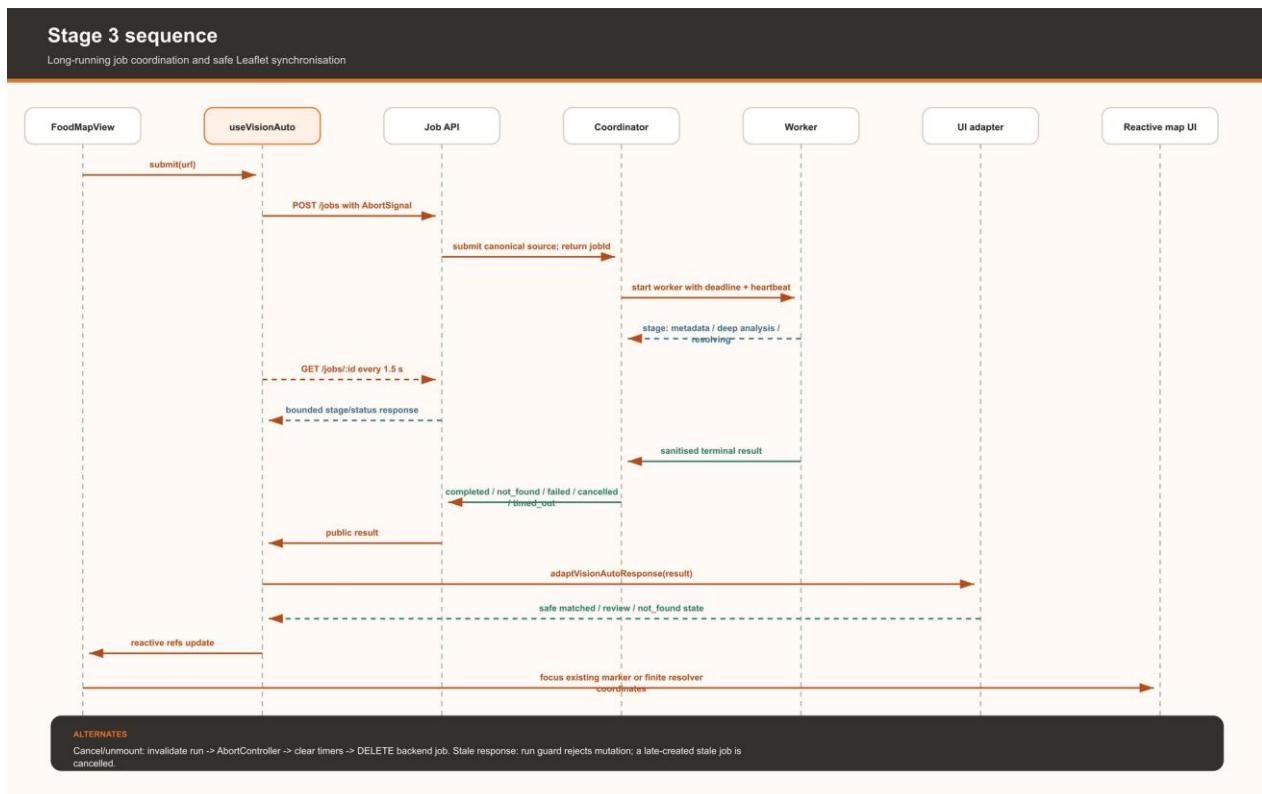


Figure 14. Coordination between FoodMapView, useVisionAuto, the asynchronous job API, coordinator/worker, response adapter and reactive Leaflet UI.

5.6.7 Why a Composable Was Selected

Option	Strength	Limitation for FoodStory
Logic inside FoodMapView	Direct access to Leaflet and stores.	A large imperative view would also own timers, HTTP, cancellation, adaptation and map lifecycle; difficult to test in isolation.
Pinia store	Shared application-wide state and development tooling.	The job is page-scoped and disposable; global state risks retaining controllers and results beyond the map lifecycle.
Generic API service	Clean HTTP wrappers and server contract.	Cannot express Vue refs, computed state, unmount cleanup, polling transitions or stale-run protection by itself.
Reusable composable	Vue-native reactivity, lifecycle cleanup and injected dependencies.	Still requires FoodMapView to retain imperative Leaflet ownership.

Table 8B. Architectural alternatives for long-running Vue state.

The composable is the strongest project-specific option because the asynchronous operation is reusable and bound to Vue lifecycle semantics, while the map itself remains deliberately outside the composable. This separation is more defensible than claiming that all complex state belongs in Pinia or that an API wrapper alone constitutes an advanced Vue technique.

5.7 Backend Boundary Supporting the Composable

The supporting backend validates sources, manages bounded jobs, cancellation, timeouts and safe terminal results, while the separate FastAPI service handles optional model inference. These components explain why the browser operation can be long-running, but they are not the assessed Vue technique. The interface-relevant contract is that every run is cancellable, every continuation is owned by the current run, and every result is adapted before it can affect Leaflet. Detailed worker,

provider and SSRF controls are summarised in Section 7 and Appendix D rather than repeated here.

5.8 Dish-First Product Pivot

The most important product decision was changing direction when exact place resolution remained unreliable within the available time, knowledge and provider budget. Some videos simply do not contain a resolvable address, and repeated OCR escalation cannot create missing evidence. Returning the highest-scoring restaurant would make a demonstration smoother but the product less trustworthy.

The final interaction separates “What dish is visually supported?” from “Which places near the current map area may serve it?” The service returns at most three bounded dish candidates. After the user chooses one, FoodStory searches local records and the configured Geoapify/OpenStreetMap-backed external provider. External results expose provider identity, address, distance and matching rationale, remain reviewable suggestions, and do not claim the original video location.

5.9 User-Facing Uncertainty States

Vision Auto does not have one binary success state. The UI distinguishes matched local place, validated external place, multiple resolved places, review-required evidence, dish candidates, dish places found, safe not_found, provider_unavailable, timeout, cancellation and operational error. provider_unavailable is a reason attached to a safe terminal state, not proof that no restaurant exists. Review-required evidence is never styled as a verified fact.

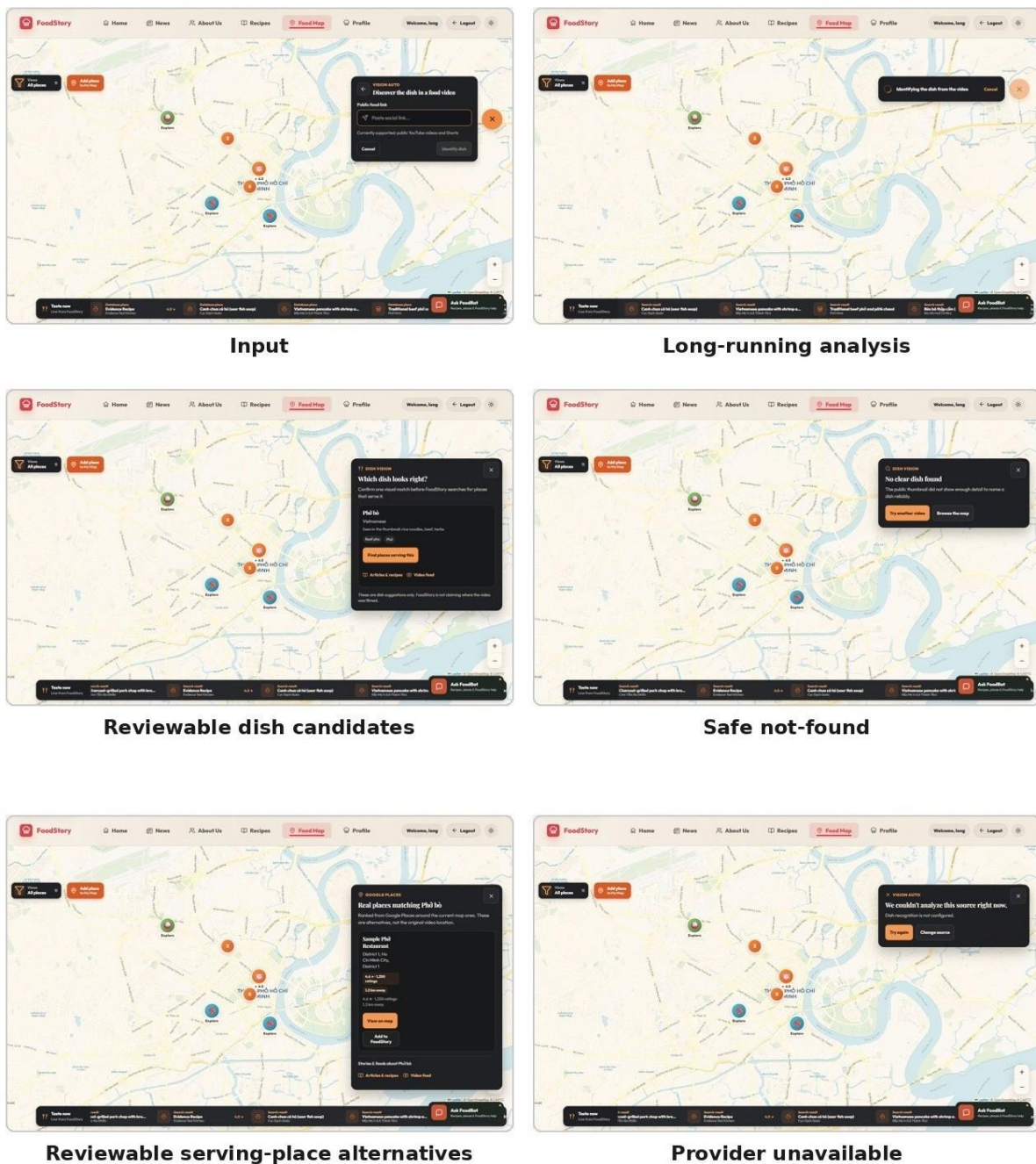


Figure 17. Vision Auto input, progress, dish candidates, safe not-found, reviewable alternatives and provider-failure states; deterministic fixture outcomes remain labelled separately from final live-provider evidence.

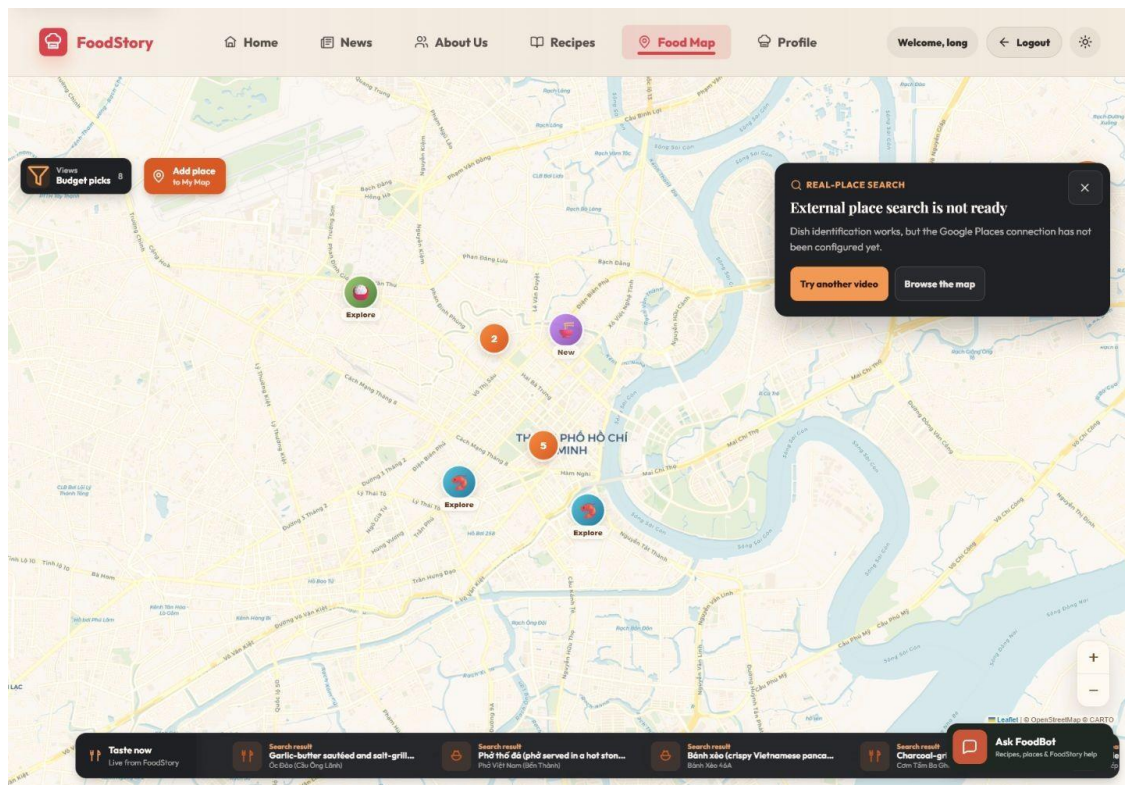


Figure 17A. Historical provider-unavailable evidence retained to demonstrate that dish identification can complete while an external serving-place provider remains unavailable and retryable.

The final Render deployment additionally demonstrates the successful provider path. After dish confirmation, FoodStory displays a bounded Geoapify and OpenStreetMap-backed nearby-place result with distance, address, matching rationale, View on map and Add to FoodStory controls. The explanatory copy explicitly distinguishes alternatives from the original filming location.

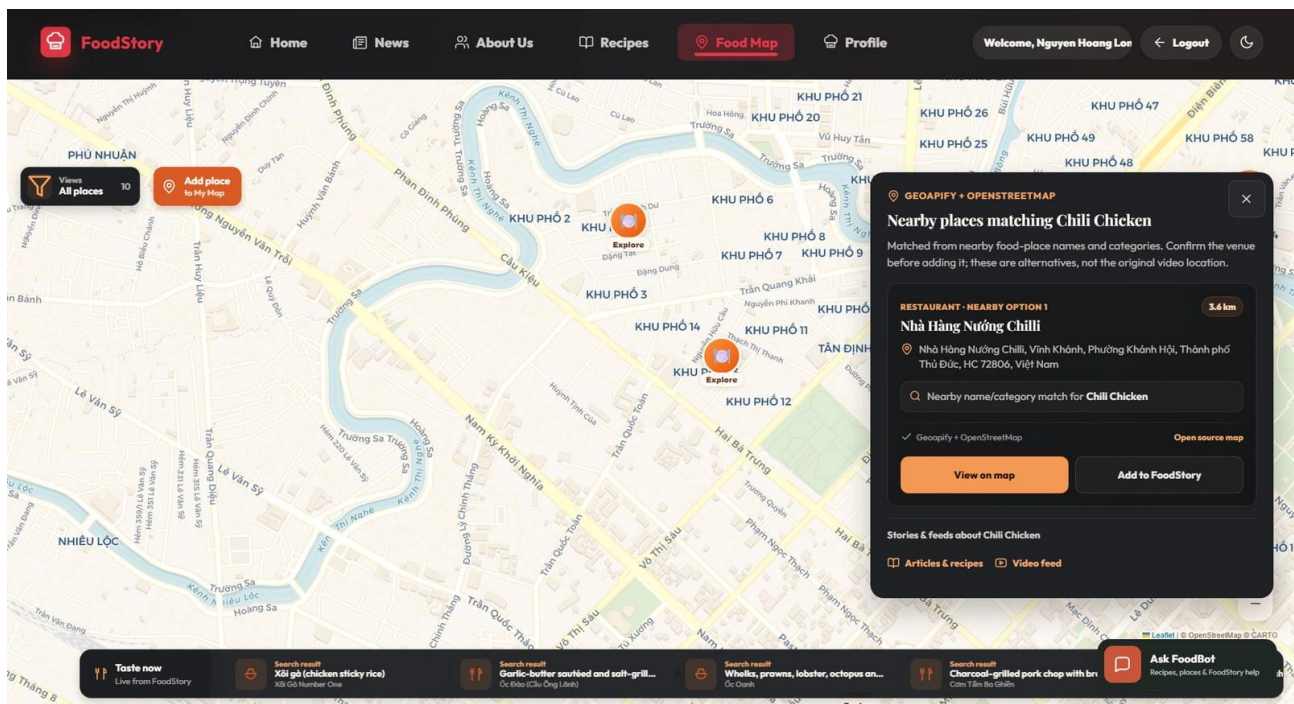


Figure 17B. Live serving-place alternative from the final deployment. Provider identity, distance and uncertainty wording remain visible before the user views or adds the place.

5.10 Failure Cases and Engineering Learning

The development history includes positive and negative evidence. One public case matched the stored Pho Le record. Another video contained an address near the fifty-third second although an earlier pipeline returned empty OCR, exposing frame-selection recall failure. Several runs retained partial street/district fragments but correctly remained review-only. A no-evidence Short returned TRACK2_V3_NO_USEFUL_VISUAL_EVIDENCE instead of a fabricated location. A heavy case produced many address-like fragments but no candidate passed the quality gate, demonstrating that evidence quantity is not corroboration.

The revision added another kind of failure evidence: tests that exposed concurrency and timer ownership defects in the Vue lifecycle. This strengthened my understanding that aborting an HTTP request is not enough; every queued continuation must also be prevented from updating current state. Production readiness therefore requires system and lifecycle testing beyond model outputs (Breck et al., 2017; Sculley et al., 2015).

5.11 Narrow Router Regression Check

A deterministic 30-case router replay was run five measured times per version after warm-up. Track 2 V3 and Final produced the same fixture-routing percentages, with only negligible timing differences. The check documents that the revised lifecycle and product contract did not weaken the existing deterministic expectations. It is a narrow software regression test, not an independent media benchmark, and it does not measure dish recognition, OCR, ASR, place accuracy, menu availability or end-to-end user latency.

Version	n	Safe routing	False promotion	Review route	Timeout	Router p50	Router p95
Track 2 V3	150	96.67%	0.00%	70.00%	0.00%	0.045200 ms	0.190700 ms
Final	150	96.67%	0.00%	70.00%	0.00%	0.048400 ms	0.194600 ms

Table 8C. Controlled Stage 3 router benchmark.

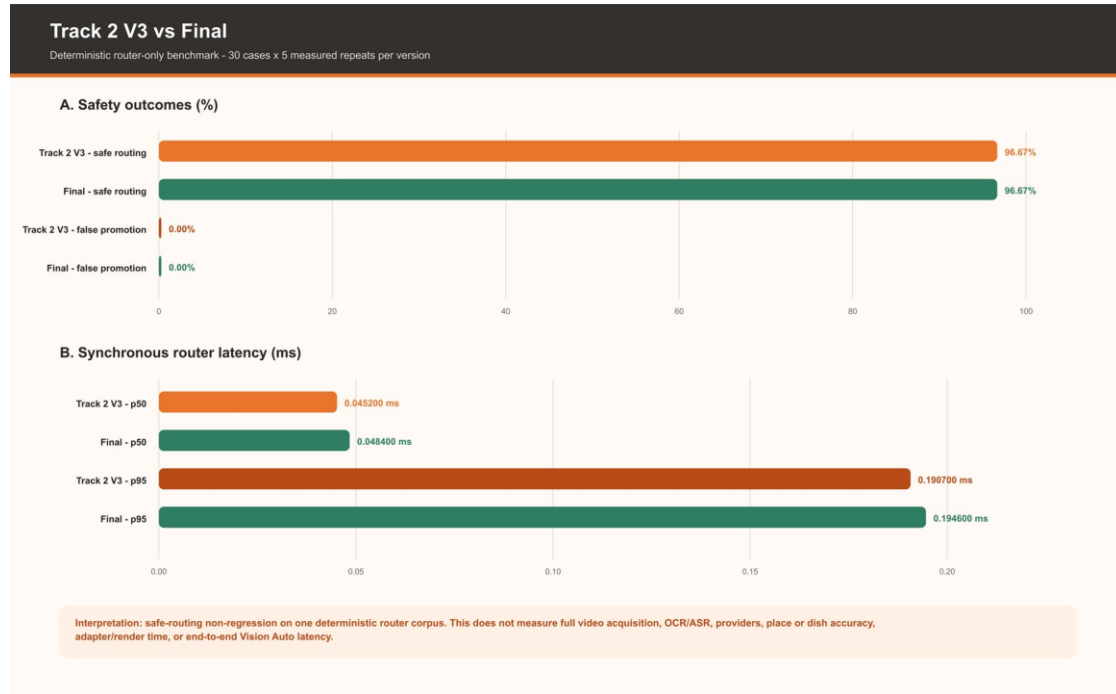


Figure 15. Narrow deterministic router regression check. The identical fixture-routing percentages document non-regression only and must not be interpreted as recognition accuracy.

Final-commit verification establishes the current source baseline through 48/48 frontend tests, a successful production build and 399/399 backend tests. These are author-run command outputs, not independent certification. Earlier focused suites for lifecycle/adaptor behaviour, API/database smoke checks, WebSocket acceptance, Vision reliability, Track 2 V3 routing, Guardian integration and AI-service safety remain useful historical evidence, but this edition does not relabel those earlier runs as final-commit reruns. Recognition quality remains a separate labelled-media evaluation requirement.

Test-to-concern matrix			
Implementation evidence - pass counts are not model accuracy			
Lifecycle / safety concern	Suite	Result	Evidence type
Invalid URL	Frontend focused	PASS	Composable unit
Submit transitions	Frontend focused	PASS	Reactive state
Poll to terminal	Frontend focused	PASS	Fake-timer lifecycle
Cancel + AbortController	Frontend focused	PASS	Signal + timer
Backend cancel request	Frontend focused	PASS	Injected API spy
State older run	Frontend focused	PASS	Deferred race
Retry identity + reset	Frontend focused	PASS	Public contract
Unmount + timer cleanup	Frontend focused	PASS	Vue lifecycle
Provider unavailable	Adapter focused	PASS	Safe adaptation
Review-required	Adapter + backend	PASS	Contract boundary
Safe not-found	Adapter + composable	PASS	Terminal safety
Dish candidates	Frontend focused	PASS	Dish-first contract
Queue/cancel/deadline	Backend reliability	31/31	Coordinator tests
Resolver/adaptor safety	Backend adapter	25/25	Boundary tests
Track 2 V3 closure	Complete selected suite	75/75	Deterministic services

Focused frontend: 10/10 | Full frontend: 29/29 | No live provider execution claimed

Figure 16. Test-to-concern matrix linking lifecycle, cancellation, stale-run, adapter and backend safety concerns to direct implementation evidence.

5.12 Limitations of the Evaluation

The router corpus contains 30 unique cases and five measured repeats, but no common labelled media ground truth for exact place or dish. Queue wait, media acquisition, frame extraction, OCR, ASR, provider, adapter, map focus and browser paint were not jointly instrumented. Live paid providers were excluded to avoid quota and network variability. The report therefore does not publish exact-place accuracy, dish top-1/top-3 accuracy, full Vision latency or Food Map FPS. Fixture screenshots remain clearly labelled as deterministic evidence.

Recognition Accuracy Status

The project did not complete a representative labelled-media accuracy study. One positive local match and several negative or review-only cases demonstrate possible outcomes, but $n=1$ cannot support an accuracy percentage. This is the central product-evaluation gap: the project demonstrates how Vue safely orchestrates uncertain output, but it does not establish how often arbitrary real videos produce the correct dish or place. A defensible follow-up should use at least 20 independently selected videos with manually established ground truth and report top-1/top-3 dish correctness, review/no-result rate, technical-failure rate and end-to-end latency. Until that study exists, the report claims lifecycle and contract reliability rather than model effectiveness.

5.13 Stage 3 Technology Decisions

Decision	Selected approach	Alternative	Project-specific evaluation
Vue state boundary	Injected useVisionAuto composable	View-local methods; Pinia store	Page-scoped reactivity and cleanup are testable without making temporary job state global.

Long task transport	Job creation + polling	Single request; SSE; WebSocket	Simple deployment, cancellation and deterministic testing; progress is coarser than push transport.
Stale-run safety	Monotonic run guard + run-local job ID	Last-response-wins	Prevents older async continuations from changing the current UI or cancelling the wrong job.
Map	Leaflet + MarkerCluster	Mapbox GL; Google Maps JS	Direct marker/layer control and open ecosystem; large imperative integration surface remains.
OCR/media	Bounded local baseline + optional providers; yt-dlp + FFmpeg	Provider-only; browser extraction	Supports deterministic paths but adds native installation and upstream-compatibility risk.
AI runtime	Separate FastAPI service	Node-only inference; hosted-only API	Good Python ML access; adds memory and deployment complexity.
Vision contract	Review and safe not_found	Always return top candidate	Protects trust even when it reduces apparent success rate.
Queue	Bounded in-memory coordinator	Unbounded execution; BullMQ/Redis	Proportionate for one academic process, not durable or horizontally scalable.

Table 9. Stage 3 technology decisions.

5.14 Personal Reflection on Stage 3

Stage 3 forced me to stop treating ambition as the same thing as a defensible result. I spent substantial time in the accuracy-frame-latency loop and initially viewed failure to identify the exact restaurant as an implementation defect that one more OCR pass might solve. Eventually I accepted that budget, knowledge, provider constraints and absent evidence set real boundaries. The stronger decision was to redesign the product contract around review and dish-first discovery.

Extracting useVisionAuto also changed how I understand Vue. At first I focused on whether the UI could call a long-running endpoint. The revision showed that the harder question is which run owns each timer, job ID and continuation. Discovering the stale create-response race and the uncanceled polling delay made the advanced technique more personal and concrete: lifecycle cleanup is not a theoretical best practice when an old result can reopen a panel or cancel the wrong backend job.

The part I am most proud of is that FoodStory can turn uncertain visual evidence into a controlled interactive workflow without pretending that review candidates are facts. The part I remain least satisfied with is proving which restaurants truly serve a recognised dish outside the local catalogue. I also learned to separate tests passed from model accuracy and router latency from full-pipeline latency. These limitations now define the next research work rather than being hidden behind a confident label.

6. Integrated Testing and Evaluation

How to read the evaluation evidence

Completion percentages are rubric-oriented evidence labels rather than predicted marks. Test counts establish exercised behaviour, while recognition quality requires labelled media. Router benchmarks cover deterministic routing only; provider, OCR, media-acquisition and browser latency are reported separately or retained as limitations. Missing baselines are recorded as unavailable rather than converted to zero.

6.1 Final Verification Snapshot

Final verification was completed on branch codex/render-deployment at commit 0473287788bad6a098f1d2724ec0e3d0c81bd412. The frontend npm ci completed successfully, the production build passed, and frontend npm test passed 48/48 tests. That command is vitest run with the configured src/**/*.*.test.js scope in jsdom. Backend npm ci completed successfully and backend npm test passed 399/399 tests. That command is node --test ./tests/*.*.test.js and covers top-level backend/tests test files rather than nested focused suites. git status --short returned no output at the authority gate, confirming a clean worktree. These are author-run local command outputs; no CI badge, independent test laboratory or external security audit is claimed. Earlier focused suites may overlap with the configured totals and are not added to them.

Area	Command/group	Passed / failed / skipped	Notes
Frontend unit/component	npm test	48 / 0 / 0	Final commit; includes current UX, geocoding and Vision regression coverage.
Frontend production build	npm run build	Passed	Final production build completed successfully; final module count and duration were not supplied.
Backend regression	npm test	399 / 0 / 0	Final npm test invocation on the clean final branch.
API/database smoke	npm run test:api	87 / 0 / 0	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.
Two-client WebSocket acceptance	npm run test:realtime	4 / 0 / 0	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.
Track 2 V3 complete	test:shorts:track2-v3-complete	75 / 0 / 0	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.
Vision Auto reliability	test:vision-auto:reliability	31 / 0 / 0	Earlier focused run;

			retained as historical evidence and not rerun after final UX/provider refinement.
Vision Auto adapter	test:vision-auto:track2-v3-adapter	25 / 0 / 0	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.
Guardian focused	node --test guardianNewsService.test.js	7 / 0 / 0	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.
AI deterministic smoke	python test_smoke.py	6 / 0 / 0	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.
Database schema audit	npm run audit:database	5 / 0 / 0	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.
Production dependency audits	npm audit --omit=dev --audit-level=high	0 high vulnerabilities	Earlier focused run; retained as historical evidence and not rerun after final UX/provider refinement.

Table 10. Final verification results.

Only 48/48 frontend tests and 399/399 backend tests are the final configured regression totals. Smaller focused suites were executed under different commands, may overlap with this coverage and are retained only by risk area. Their counts must not be added to or compared directly with the configured final totals.

6.1.1 Test-to-Risk Interpretation

The verification results are read as layered risk evidence. Frontend tests protect navigation, Food Map access, bounded chat memory and Vision lifecycle adaptation. Backend discovery tests emphasise negative OCR evidence, provider optionality, authentication, remote-image safety and FoodBot routing. Live scripts exercise database constraints and two-client WebSocket delivery, while focused Vision tests guard against stale results, timers surviving cancellation, automatic promotion of review candidates and incorrect provider-failure mapping. The AI smoke covers six deterministic runtime and security controls.

6.2 Corrective Test History

Three first failures were retained because they improved the evidence process. Frontend lock validation found missing package-lock entries and was repaired with a lockfile-only install. The first two-client fixture omitted a required recipe image URL and failed before socket assertions; correcting it enabled the complete 4/4 acceptance. The first database audit encountered an older migration ledger without applied_at, prompting a compatible audit harness and a 5/5 rerun.

6.3 Strengths and Gaps

Backend and Vision coverage is strongest around negative outcomes, ownership, provider optionality, timeout, queue, cleanup and safe closure. FoodBot adds routing, prompt-budget and bounded-memory tests, and the two-client sequence establishes integrated real-time delivery. Frontend automation remains focused rather than exhaustive across browsers, visual states and assistive technologies. Paid-provider success paths were exercised through injected contracts and failure handling; the final screenshot run also retained the genuine FoodBot MIME-upload failure as a limitation.

Three bounded omissions remain: a destructive fresh-schema migration was avoided on the shared database, full OpenCLIP inference was not forced on the final CPU environment, and ban/unban was not repeated against shared seeded accounts whose engagement data would be removed.

6.4 Quality-Attribute Evaluation

Attribute	Evidence	Assessment	Remaining risk
Functionality	Rubric matrix, browser flows and API checks	Strong	Public Render deployment and the final source archive are verified; the Stage 3 video URL remains [STAGE_3_VIDEO_URL_TO_BE_INSERTED].
Reliability	Timeouts, cancellation, stale guards, negative tests	Strong	External provider and network variability.

Attribute	Evidence	Assessment	Remaining risk
Security	JWT/bcrypt, ownership, Helmet/CORS/rate/body limits, SSRF connected-peer checks and authenticated AI-service boundary	Strong academic baseline	Browser token storage, process-local counters, demo seed credentials and no penetration test.
Performance	Controlled build/Lighthouse/API/WebSocket benchmark; bounded jobs; final WebP asset remediation	Measured, mixed and acted upon	Recipe-detail p95 regressed; full Vision latency and map FPS remain unmeasured.
Accessibility	Skip link, route title/focus, Escape/focus restoration, reduced-motion handling, labelled status/alert regions and 390 px no-overflow observation	Moderate and explicitly evidenced	No formal WCAG conformance claim or assistive-technology user study.
Maintainability	Domain stores/services, extracted composable, evidence docs	Moderate-strong	FoodMapView and provider matrix remain complex.
Usability	Polished light/dark UI, deployed workflows and one informal formative usability review.	Improved and directly evidenced	Only one participant; no structured usability study or significant production traffic.
Reproducibility	Final branch codex/render-deployment, commit 04732877, clean authority-gate worktree, Render deployment, 48/48 frontend tests, successful build and 399/399 backend tests.	Strong source identity and verified packaging	Archive hash, reopen, secret scan and clean extraction verified; Stage 3 video URL remains [STAGE_3_VIDEO_URL_TO_BE_INSERTED].

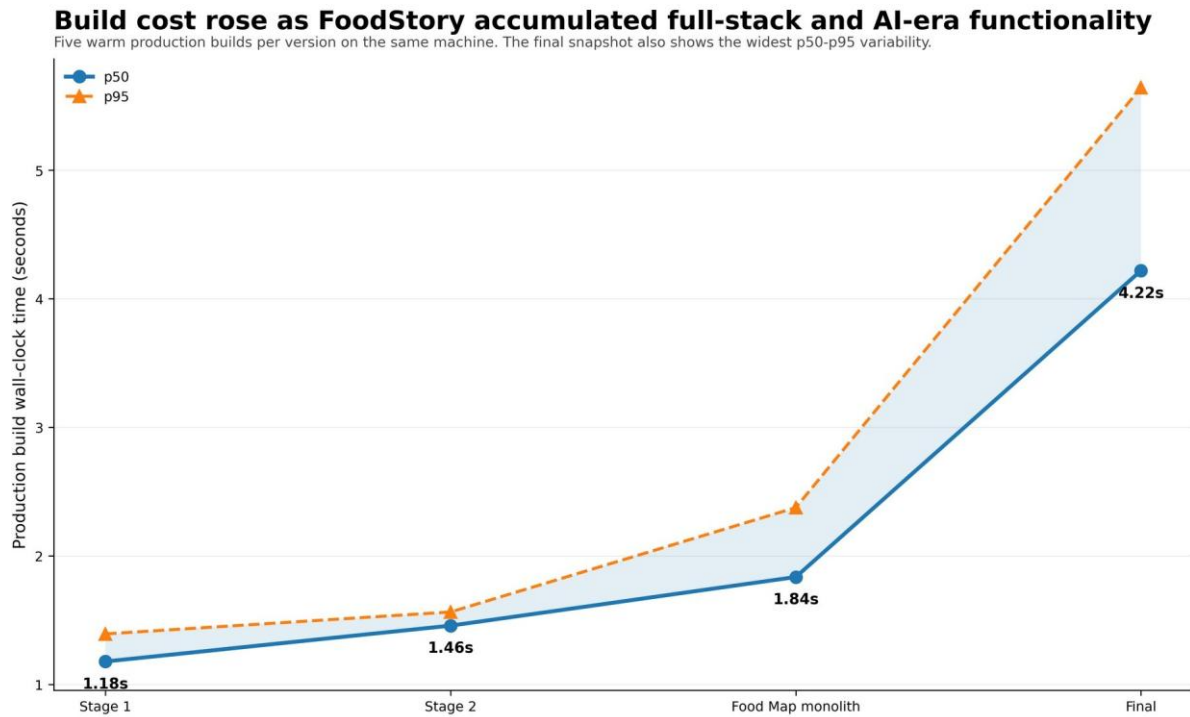
Table 11. Quality-attribute evaluation.

6.5 Controlled Comparative Performance Evaluation

A separate controlled benchmark compared feature-specific Git snapshots and retained 1,670 raw observations across build, Lighthouse, navigation, responsive, API, real-time and Vision-router measurements. Builds and Lighthouse used one warm-up plus five measured runs where supported; API used 30 sequential requests; real-time delivery used ten iterations per event type at 1, 5 and 10 viewers (FoodStory, 2026c).

6.5.1 Build and Frontend Route Performance

The controlled benchmark captured the pre-remediation snapshot. Warm production-build p50 rose from 1.18 seconds in Stage 1 to 4.22 seconds, p95 reached 5.64 seconds and generated output reached approximately 516.4 MB, dominated by photographic assets. This finding triggered the historical WebP remediation: 200 recipe photographs were converted from PNG to WebP and the then-current image directory fell by 89.83%. This performance history is retained without reusing any obsolete archive size or hash as final-package evidence.



Static assets dominate the final production package

The output-size regression is primarily an asset-management problem, not evidence that JavaScript alone reached 516 MB.

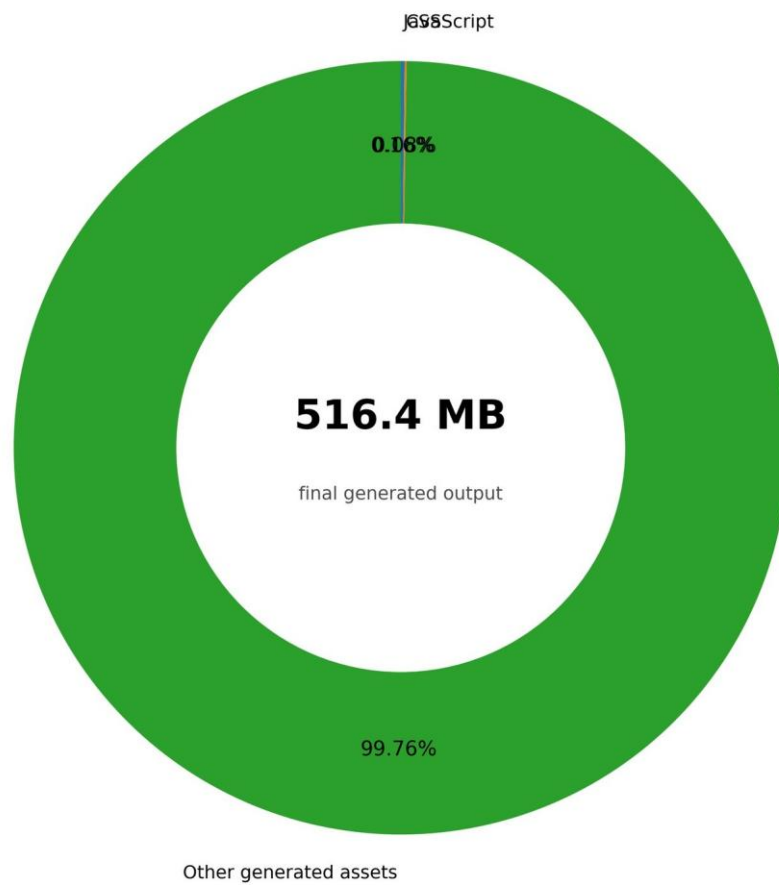


Figure 18. Pre-remediation production-build evolution and output composition. The static-asset regression identified here was corrected by the final WebP source-freeze described in Section 6.5.1.

Frontend performance remained route-specific. Final Home LCP improved substantially from Stage 2 on desktop and mobile and was also below the Stage 1 desktop baseline, while Recipes and Recipe Detail showed smaller regressions in several Stage 2-to-Final comparisons. The evidence therefore supports route-level conclusions rather than one application-wide speed claim.

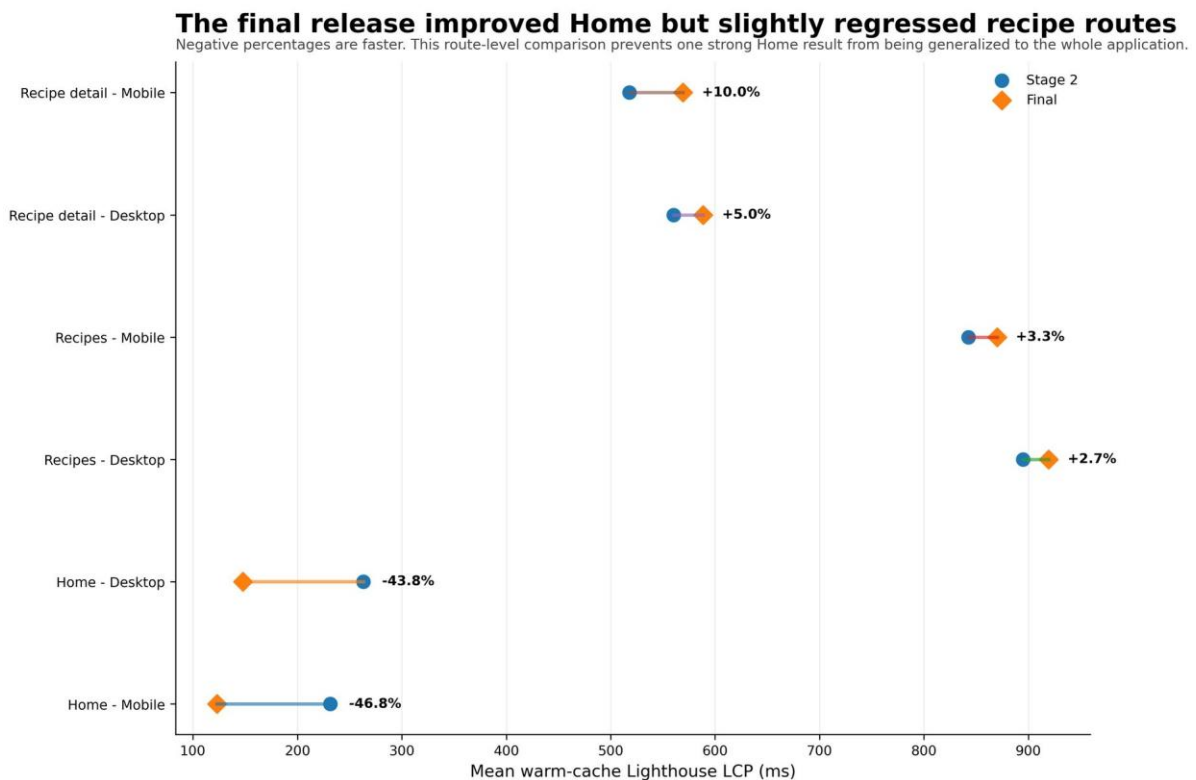
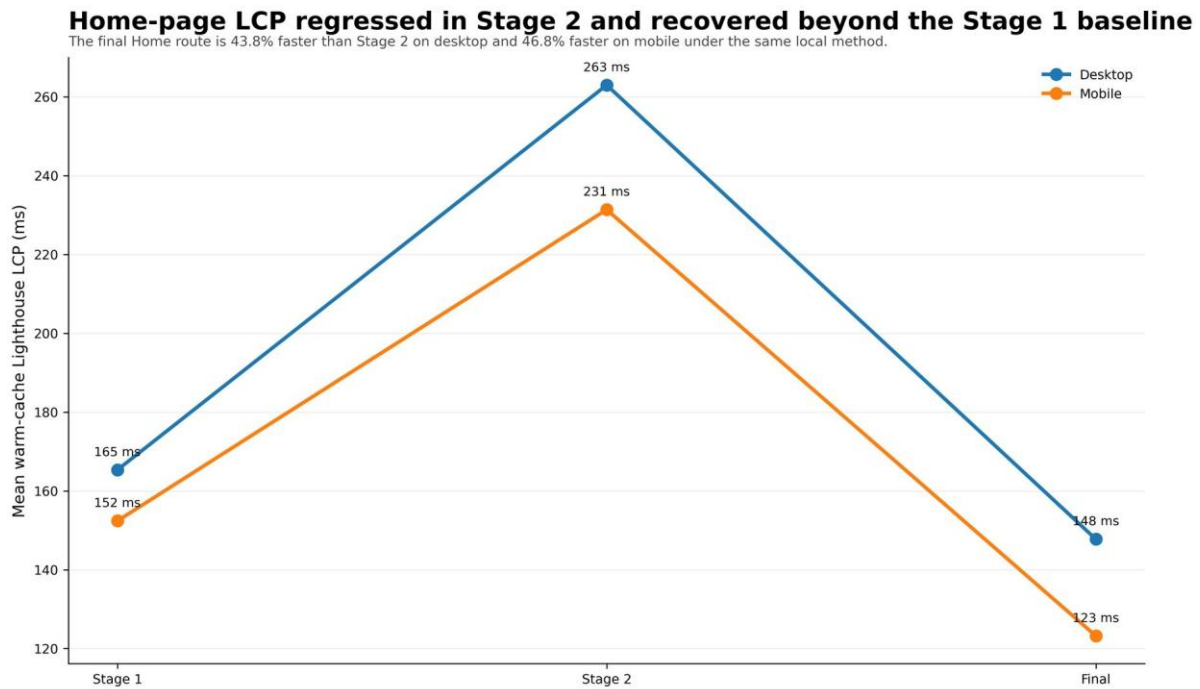


Figure 19. Warm-cache Lighthouse LCP across versions, routes and viewports. Home improved substantially in Final, while recipe routes show smaller regressions.

6.5.2 API and Real-Time Delivery

Thirty sequential loopback requests were measured for shared login and read-only endpoints. Median API latency remained broadly stable. Recipe Detail improved from 51.06 ms to 44.26 ms at p50, while

p95 increased from 144.07 ms to 217.63 ms. Typical responsiveness improved in this run, but the less predictable tail should be repeated before it is treated as stable.

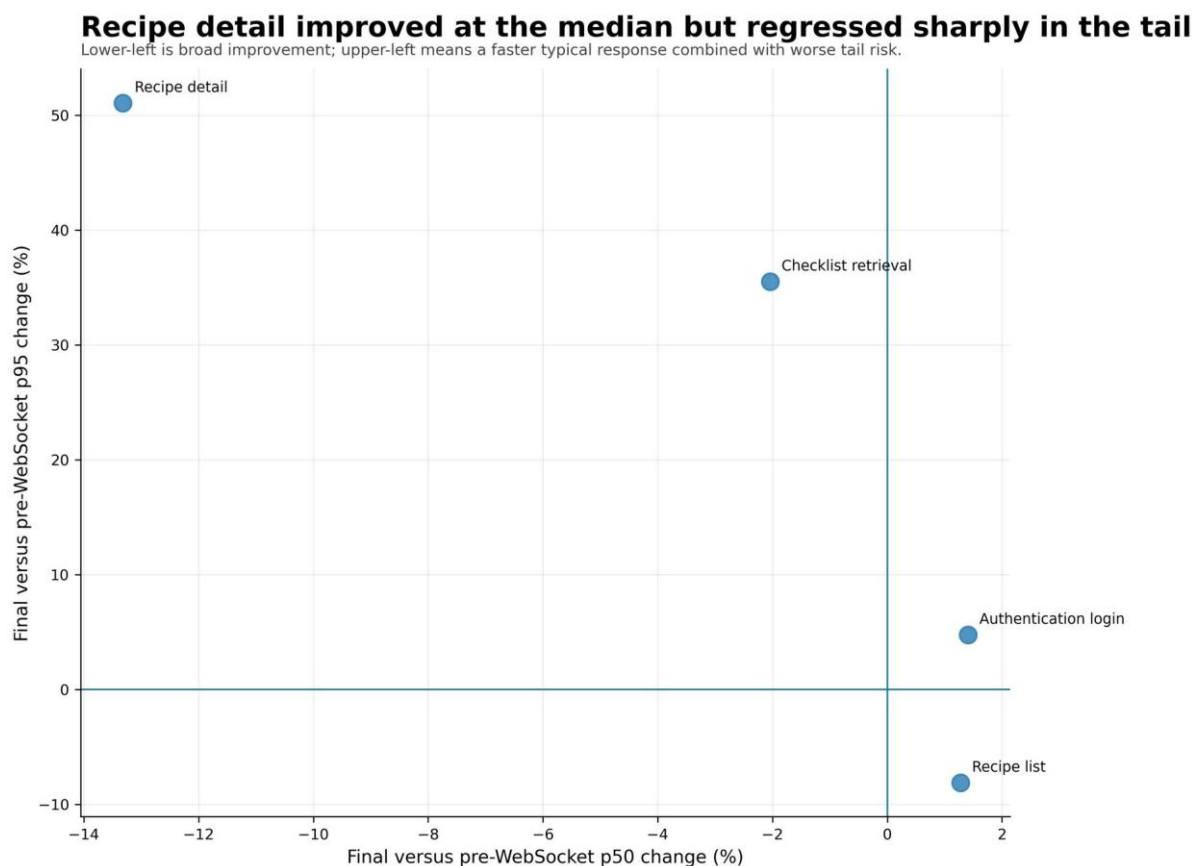
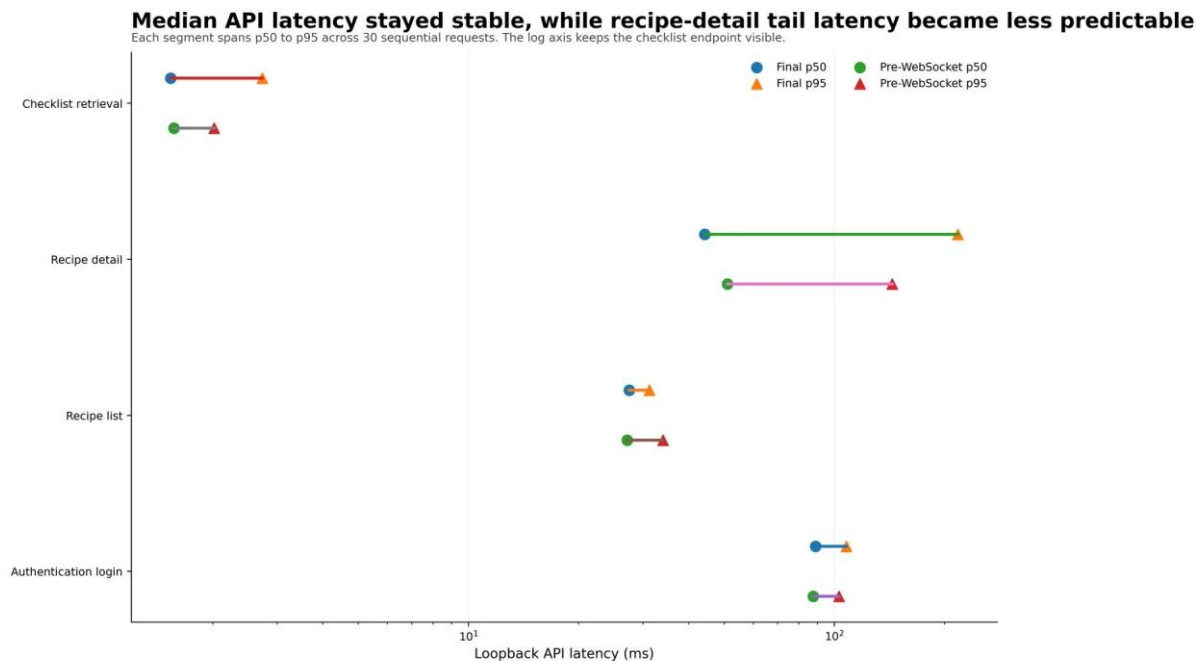


Figure 20. API p50-p95 intervals and median-versus-tail change. Recipe Detail demonstrates why a faster median does not guarantee a better p95.

Because the pre-WebSocket snapshot had no broadcast mechanism, no baseline event-latency value exists. Final retained 640 request-to-WebSocket-receive observations across comment create/edit/delete and rating update at 1, 5 and 10 viewers. Median delivery remained close to 10 ms,

with zero lost, duplicate or out-of-order events. Browser store-update and paint timing remain future instrumentation work.

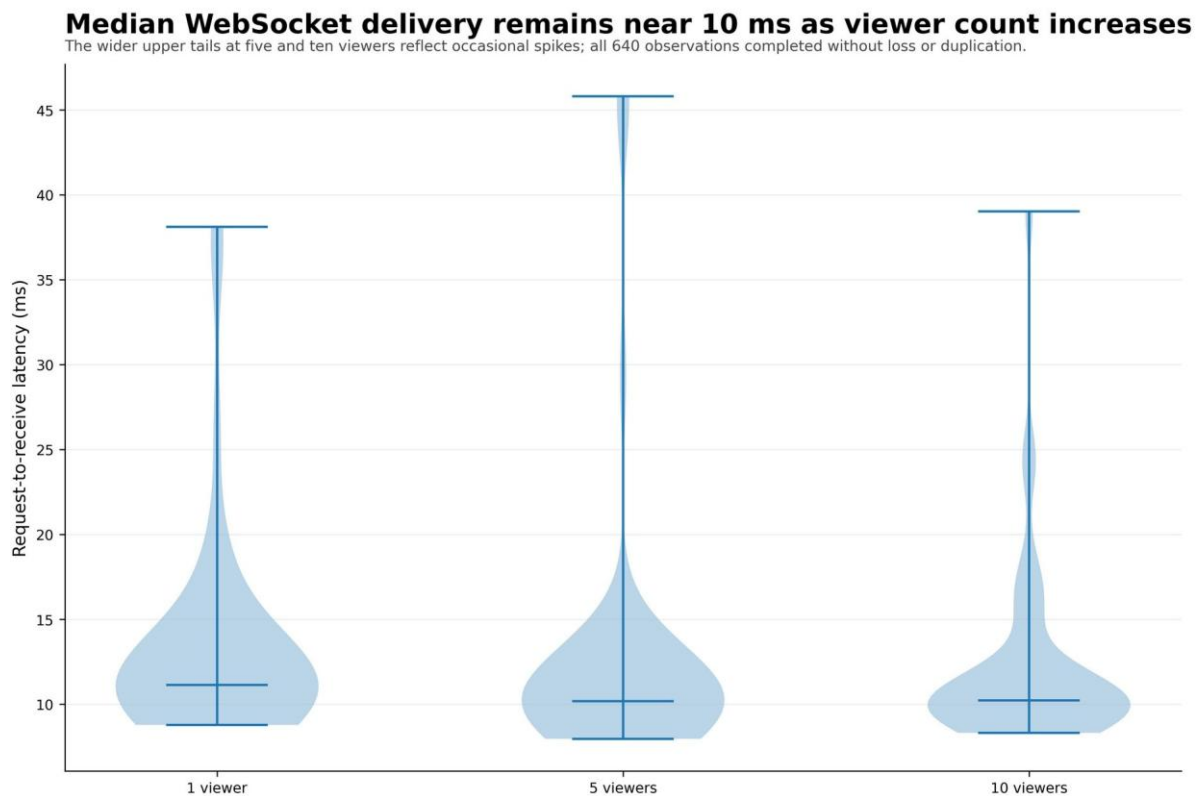
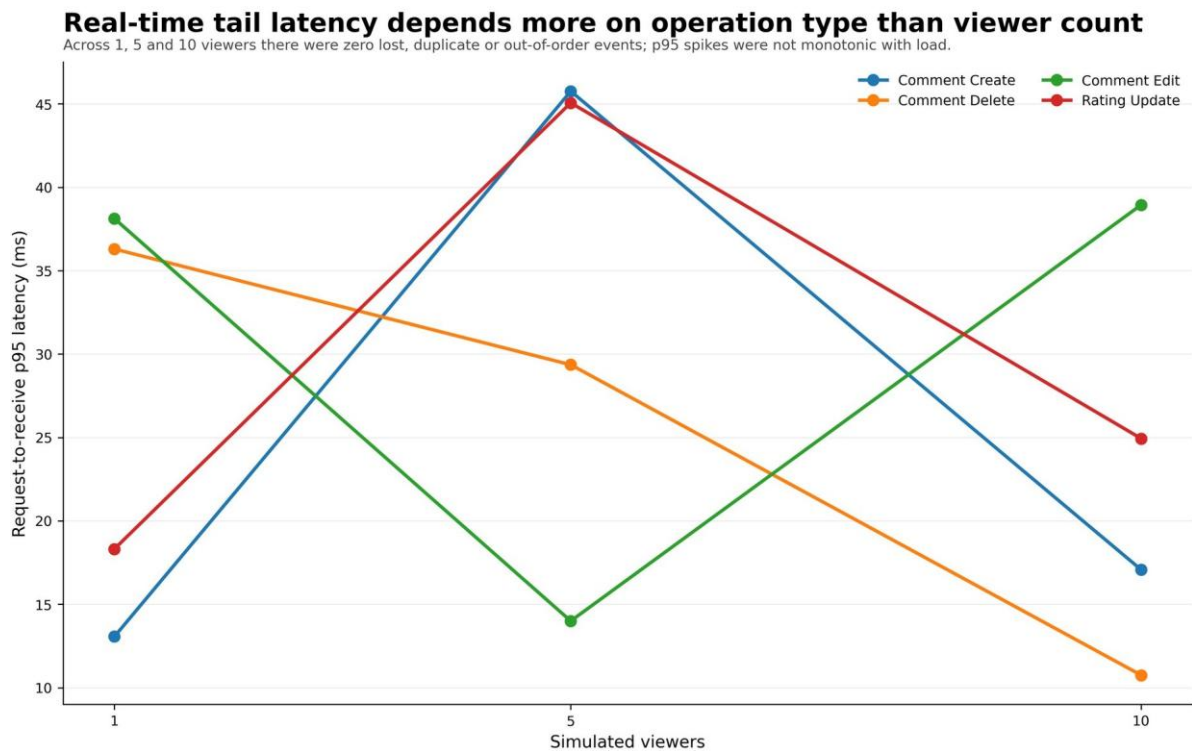


Figure 21. Final real-time request-to-receive latency by operation and viewer count. The distributions include 640 observations with no loss, duplication or reordering.

6.5.3 Responsive Structural Audit

The deterministic responsive audit compared Home, Recipes and Recipe Detail at six viewport widths. Most cells show fewer detected structural issues in Final than Stage 2, particularly clipped text and

undersized touch targets. It is a repeatable DOM/layout heuristic; real-user satisfaction remains unmeasured.

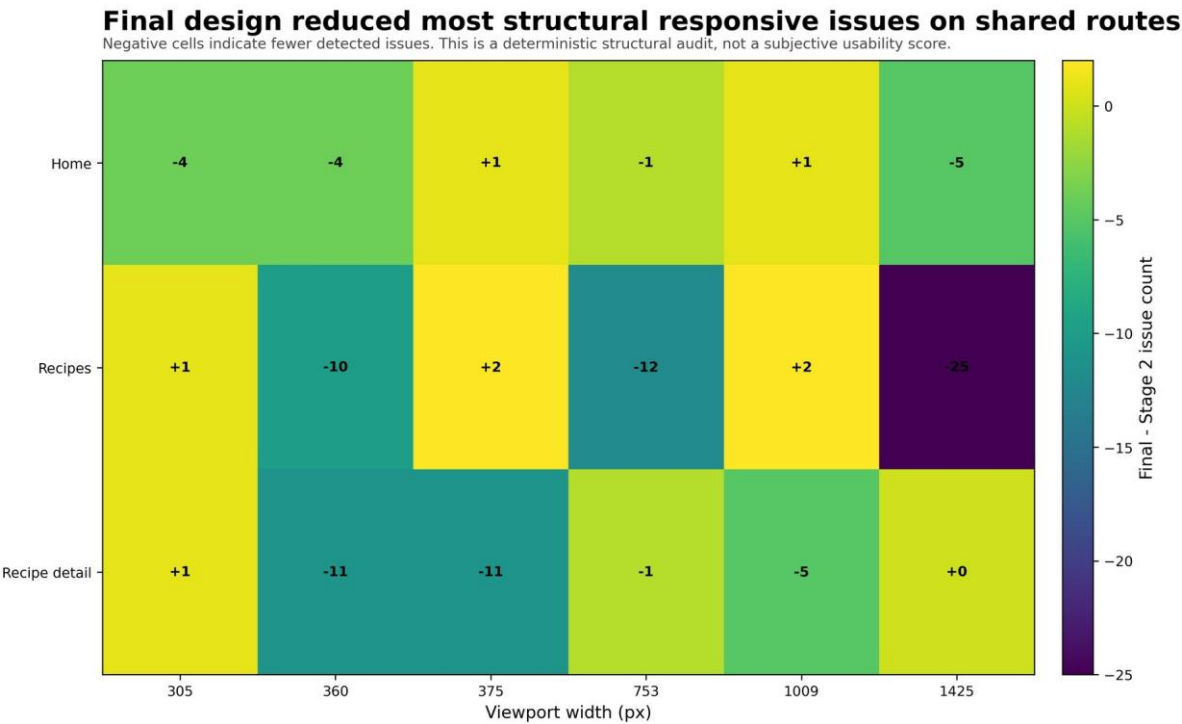


Figure 22. Final-minus-Stage-2 responsive issue changes by route and viewport. Negative cells indicate fewer detected structural issues.

7. Security, Accessibility, Privacy and Responsible AI

7.1 Security Review

The security review distinguishes application assurance from packaging evidence.

FoodStory_Final_Source_04732877.zip was generated directly from commit

0473287788bad6a098f1d2724ec0e3d0c81bd412 with the report-input DOCX explicitly excluded. ZIP integrity, every-entry reads, safe paths, forbidden/sensitive-file and likely-secret scans, clean extraction and expected frontend/backend/documentation structure all passed. The archive contains 814 entries (767 files and 47 directories), is 55,496,926 bytes (52.925993 MiB), and has SHA-256 3d66a5f34ac2bbc854c0ce4104c89a85a523385b33216bf4cc60b5d0d2d23e1d. These checks establish package hygiene and reproducibility; they do not constitute penetration testing or recognition-accuracy evidence.

The public Render deployment demonstrates the application but is not a substitute for clean submission packaging. The previously uploaded backend ZIP contained a real .env file and must not be submitted. The final package must exclude environment files, service credentials, node_modules, virtual environments, caches, build outputs and downloaded media; any exposed provider keys should be rotated.

7.1.1 Final Security Hardening

FoodStory uses layered controls because authentication, uploads, remote URLs and AI inference cross different trust boundaries. Express applies Helmet, an origin allowlist, a 1 MB JSON limit, a general 500-request/15-minute limit, a stricter 25-attempt authentication limit, required JWT configuration and database-backed role/ban checks. Multipart and AI-image paths cap content at 5 MB and validate type and decoding. Remote-image and social-source flows reject unsafe protocols, private DNS results and redirects, then revalidate the actual connected peer. Remaining risks include process-local counters, browser token storage, local demo credentials and the absence of a penetration test.

7.1.2 AI Service Trust Boundary

The FastAPI AI service is isolated because model dependencies, image decoding and remote acquisition have a different risk and resource profile from the main API. Express is the intended caller and propagates a server-only bearer token. In production mode FastAPI fails closed when no token is configured, compares supplied tokens using a constant-time operation and applies a separate default limit of 120 embedding requests per minute. Text inputs are bounded to 4,000 characters, URLs to 2,048 characters, redirects to three and images to 5 MB. The 6/6 smoke result covers deterministic boundary controls; visual-model accuracy and production throughput remain separate evaluation questions.

7.2 Accessibility and Interaction Safety

Accessibility is implemented through cross-cutting application behaviour and local component semantics. The global shell includes a skip link, a focusable main landmark, route-specific page titles and focus transfer after navigation. The final UX refinement adds a persistent Recipe guide, visible active-section state, explicit Show/Hide controls, bounded comment loading, reduced-motion behaviour and address-search feedback. Collapsible controls should expose aria-expanded, active navigation should not depend on colour alone, and loading/no-result/provider-error messages should remain available to assistive technology.

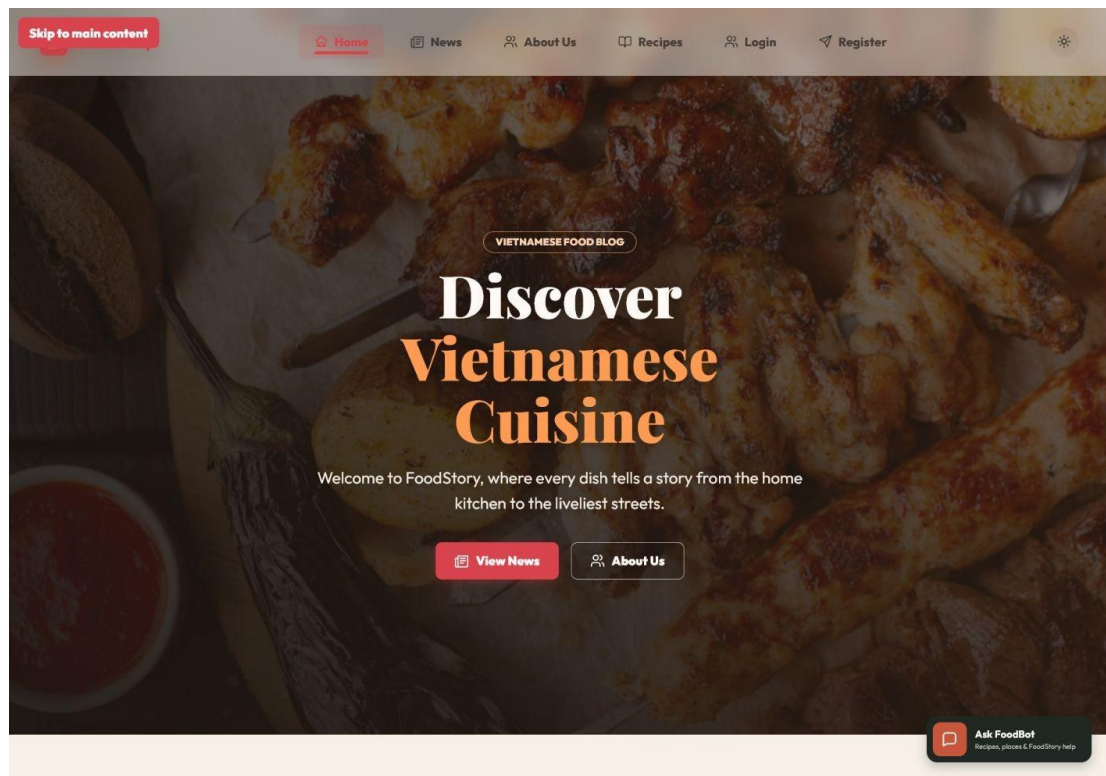


Figure 22A. The global shell exposes a visible Skip to main content control when keyboard focus reaches the skip link; this screenshot proves that specific mechanism rather than complete accessibility conformance.

7.3 Authentication and Data Protection

Passwords are hashed with bcrypt and excluded from safe user objects. Parameterised queries protect SQL structure, and role/owner checks are enforced on the server. JWT claims authenticate API and WebSocket requests, while browser localStorage introduces an XSS-related trade-off. A production deployment should compare short-lived access tokens, secure same-site cookies, refresh rotation and revocation against the current academic baseline.

7.4 Privacy and External Media

Vision features process public URLs and bounded images, but public availability does not eliminate privacy concerns. The pipeline redacts source details in logs, avoids returning raw OCR dumps or provider credentials to the browser and keeps uncertain candidates reviewable. Address and place-name searches may be transmitted to Geoapify or an OpenStreetMap-backed geocoding service. FoodStory does not silently persist a provider result: the user reviews the returned candidate, confirms or adjusts the final pin and then chooses whether to save it.

7.5 Responsible AI Contract

FoodStory's responsible-AI design is expressed through interface and system contracts: the application states what it can analyse, communicates long-running progress, allows cancellation and retry, distinguishes model/provider failure from lack of evidence, and asks for user confirmation before searching places for a selected dish. Review-required results are not styled as verified facts. This is an implementation of scoped service and efficient correction rather than a generic ethics statement (Amershi et al., 2019).

NIST (2023) frames trustworthy AI across validity, safety, security, transparency and privacy; FoodStory uses not_found rather than unsupported inference.

8. Critical Discussion and Personal Reflection

8.1 What Worked Well

FoodStory's greatest strength is progressive coherence. Stage 1 interactions became the public foundation of a larger system. The recipe catalogue supports a complete journey; real-time interactions are anchored in database consistency; moderated contributions move through visible cross-role states; FoodBot exposes grounded local sources and bounded context; and the Food Map gives the project an identity beyond generic CRUD. The advanced Vision technique is integrated into this meaningful product surface with explicit lifecycle and uncertainty handling.

The evidence process also improved the product. Security review removed tracked caches and unsafe seed logging; later hardening added Helmet, request limits, AI-service authentication and connected-peer validation. Tests exposed lockfile, fixture, audit-harness, stale-run and timer-ownership defects. An earlier optimisation pass converted 200 recipe photographs to WebP and substantially reduced that snapshot's package size. Because the final deployment contains later provider and UX changes, its submission archive still requires a fresh packaging and scan step. These less visible refinements make the project more reproducible, deployable and defensible.

8.2 What Remains Weak

The project still lacks significant real-user traffic and a structured usability study with representative guest, user and administrator participants. One informal formative review was completed before finalisation. The participant identified unrelated collections during recipe search, excessive Recipe Detail scrolling, long comment lists, uncomfortable motion and inefficient manual map pinning. The review acted as a trigger rather than a complete validation study: each issue was also checked through direct observation of page length, interaction steps, responsive layout and accessibility semantics before implementing focused search, sticky section navigation, collapsible content, bounded comment loading, reduced-motion behaviour and address-assisted place selection. Because the review involved one participant and no formal task protocol, it supports iterative refinement but not generalisable usability claims.

The Admin Dashboard is complete for individual moderation and governance, but a production operator workspace would still require queue priority, audit history, bulk actions and stronger high-volume table alternatives. Process-local WebSocket rooms, the Vision queue, provider cache and rate counters cannot scale consistently across backend instances. The most important academic gap is recognition evaluation: the report contains one positive local match and several failure/review cases, but no representative labelled-video study from which dish or exact-place accuracy can be calculated. FoodBot visual search also lacks a successful final browser-inference capture, and CPU/provider latency remains variable. The Render deployment is live and clean source-archive packaging is verified. The face-present Stage 3 video URL is the sole remaining submission action: [STAGE_3_VIDEO_URL_TO_BE_INSERTED].

8.3 Maintainability and Technical Debt

FoodMapView grew too large during rapid development. Later extraction of FoodMapDiscoveryTray, FoodMapImportPanel, FoodMapRail, VisionAutoResultPanel and useVisionAuto reduced the view substantially, but it remains a complex integration surface. This reflects the hidden technical debt described in machine-learning systems: provider dependencies, data/evidence contracts and configuration can create maintenance cost beyond the model code (Sculley et al., 2015).

In hindsight, I would define tests earlier, preserve stage evidence, and queue OCR/ASR work.

8.4 Reflection on AI-Assisted Development

I used ChatGPT and Codex for more than wording suggestions. They proposed implementation changes, tests and report edits, sometimes across several files. That saved time, but it also made review discipline essential because convincing output could still be wrong or outside the assessment focus. I inspected diffs, ran the affected flow and kept or reverted changes rather than treating an agent response as approval. The clearest warning was a script that appeared to accept a live URL but actually exercised mocked fixtures only.

For the oral defence, I need to explain the implementation rather than rely on the report: how the run guard differs from AbortController, why polling cleanup matters, how the adapter blocks unresolved candidates and where server-side trust boundaries begin. AI assistance accelerated development, but understanding, verification and responsibility are the basis on which I submit the work.

The report therefore separates final-commit checks from historical suites and deterministic fixtures. This distinction is intended to prevent a narrow or earlier test from being presented as broader evidence than it actually provides.

9. Future Development

9.1 Product and UX Priorities

Priority	Proposed work	Level
Structured usability evaluation	Recruit representative guest, user and administrator participants; measure task success, navigation errors and comprehension of AI uncertainty.	High
Admin redesign	Create an action-prioritised moderation workspace with audit history, bulk operations and responsive table alternatives.	High
Serving-place evidence	Collect structured restaurant-menu data and community confirmation instead of relying only on broad place text search.	High
International expansion	Use locale-aware dish aliases, multilingual prompts and region-specific provider settings.	Medium-high
Platform support	Add Instagram/Facebook/TikTok only where acquisition terms and reliable public contracts permit.	Medium
Final submission packaging	Retain the verified source archive metadata with the submission, record and upload the face-present Stage 3 video, replace [STAGE_3_VIDEO_URL_TO_BE_INSERTED] with its URL, and confirm Mercury compatibility only if specifically required.	High

Table 12. Priority future work.

9.2 Technical Scaling

A production backend should replace process-local WebSocket rooms and the Vision queue with shared infrastructure such as Redis pub/sub and BullMQ. Job metadata and results should be durable across restarts. Provider requests require central rate limiting, cost budgets and observability. Media processing should run in isolated workers with explicit storage lifecycle policies. Frontend state could adopt a server-cache/query abstraction to reduce cross-store duplication.

9.3 Vision Research Direction

The long-term Vision goal is evidence-backed, multi-country dish and serving-place discovery. Future evaluation should measure dish precision, useful-frame recall, address accuracy, latency and user confirmation. Community signals require provenance and moderation, and the system must keep “dish shown”, “original location” and “places that may serve the dish” distinct.

10. Conclusion

FoodStory demonstrates progressive development across all three COS30043 stages. Stage 1 created a responsive Vue identity and the required Home, News and About interactions. Stage 2 transformed the prototype into a persistent full-stack platform with authentication, a detailed recipe ecosystem, authorised contribution, operational Admin governance, real-time social updates and the grounded FoodBot extension. Stage 3 added progressive-access Food Map discovery and an advanced Vue composable that coordinates long-running vision-assisted work.

The strongest technical outcome is the relationship between usability and correctness. Real-time updates are broadcast after persistence; ownership and Admin authority are enforced on the server; FoodBot separates stored facts, private data and optional external knowledge; long-running jobs can be cancelled and cannot overwrite newer results; and uncertain visual evidence is represented as review or provider limitation rather than false resolution. The dish-first pivot is a more defensible contract created through experimentation and critical reflection.

The project remains limited by the absence of a structured multi-participant usability study, formal accessibility conformance testing, multi-instance realtime/job infrastructure and a representative labelled-video accuracy benchmark. External place results remain discovery suggestions rather than verified menu evidence. The final Render deployment, UX evidence, clean authority-gate worktree, author-run test baseline and verified source archive strengthen the submission, but they do not constitute independent certification and do not replace the missing recognition study. The Stage 3 video URL remains [STAGE_3_VIDEO_URL_TO_BE_INSERTED].

References

External academic and technical sources

1. Amershi, S. et al. (2019) "Guidelines for Human-AI Interaction", Proceedings of CHI 2019, doi:10.1145/3290605.3300233.
<https://doi.org/10.1145/3290605.3300233>
2. Angular (2026) What is Angular? <https://angular.dev/overview>
3. Bootstrap Team (2026) Grid system - Bootstrap 5.3. <https://getbootstrap.com/docs/5.3/layout/grid/>
4. Breck, E. et al. (2017) "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction", IEEE Big Data.
<https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>
5. COS30043 Interface Design and Development (2026) Custom Web Application Project Brief. Swinburne University of Technology.
6. Express.js (2026) Using middleware. <https://expressjs.com/en/guide/using-middleware/>
7. FastAPI (2026) FastAPI documentation. <https://fastapi.tiangolo.com/>
8. IETF (2015) RFC 7519: JSON Web Token (JWT). <https://www.rfc-editor.org/rfc/rfc7519>
9. Leaflet Contributors (2026) Leaflet API reference. <https://leafletjs.com/reference.html>
10. MDN Web Docs (2026) WebSocket. <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
11. Microsoft Research (2019) Guidelines for Human-AI Interaction project.
<https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
12. MongoDB (2026) Data modelling. <https://www.mongodb.com/docs/manual/data-modeling/>
13. NIST (2023) Artificial Intelligence Risk Management Framework 1.0. <https://airc.nist.gov/airmf-resources/>
14. OpenStreetMap Foundation (2026) Tile usage policy. <https://operations.osmfoundation.org/policies/tiles/>
15. OWASP Foundation (2021a) SQL Injection Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
16. OWASP Foundation (2021b) Server Side Request Forgery Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
17. OWASP Foundation (2021c) Authentication Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
18. Pinia (2026) Introduction and core concepts. <https://pinia.vuejs.org/introduction>
19. Radford, A. et al. (2021) "Learning Transferable Visual Models From Natural Language Supervision".
<https://arxiv.org/abs/2103.00020>
20. React (2026) Thinking in React. <https://react.dev/learn/thinking-in-react>
21. Reimers, N. and Gurevych, I. (2019) "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks".
<https://arxiv.org/abs/1908.10084>
22. Sculley, D. et al. (2015) "Hidden Technical Debt in Machine Learning Systems", NeurIPS. <https://research.google/pubs/hidden-technical-debt-in-machine-learning-systems/>
23. Tailwind CSS (2026) Responsive design. <https://tailwindcss.com/docs/responsive-design>
24. The Guardian (2026) Open Platform. <https://open-platform.theguardian.com/>
25. TheMealDB (2026) API. <https://www.themealdb.com/api.php>
26. Vite (2026) Why Vite. <https://vite.dev/guide/why.html>
27. Vue Router (2026) Navigation guards and Composition API. <https://router.vuejs.org/guide/advanced/navigation-guards.html>
28. Vue.js (2026a) Composables. <https://vuejs.org/guide/reusability/composables>
29. Vue.js (2026b) Composition API FAQ. <https://vuejs.org/guide/extras/composition-api-faq.html>
30. W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>
31. yt-dlp Contributors (2026) yt-dlp documentation. <https://github.com/yt-dlp/yt-dlp>

Project evidence artefacts (self-authored; not independent sources)

11. FoodStory (2026a) Final source snapshot, branch codex/render-deployment, commit 0473287788bad6a098f1d2724ec0e3d0c81bd412, 27 July 2026.
12. FoodStory (2026b) Deep Academic Report Audit, Final Screenshot Evidence and verification records, 21 July 2026.
13. FoodStory (2026c) Controlled Comparative Benchmark Report and retained raw benchmark data, 20 July 2026.
14. FoodStory (2026d) Final deployment and UX-refinement evidence set, Render deployment <https://foodstory-cos30043-hoanglong212.onrender.com>, branch codex/render-deployment, commit 0473287788bad6a098f1d2724ec0e3d0c81bd412, 27 July 2026.

Appendix A - Rubric Evidence Matrix

The matrix below maps the assessment brief to the final evidence. Historical Stage 1 evidence and final evolved functionality are intentionally separated.

Stage / criterion	FoodStory evidence	Status / limitation
Stage 1 - project setup	Vue 3, Vite, Router and required views.	Complete.
Stage 1 - Home/About/responsive	Home visual identity; reactive greeting and Food Mood; desktop and phone evidence.	Complete; no formal user study.
Stage 1 - News	Historical 12-item JSON, full-field search and four-per-page pagination.	Complete historical requirement; final source uses Guardian proxy.
Stage 2 - technical	Components, directives, arrays, forms, validation, methods/computed, pagination, API and responsive design.	Complete; accessibility audit remains partial.
Stage 2 - functional	Auth, roles, search/filter, social functions, authorised CRUD and MySQL persistence; final source plus historical focused integration runs.	Implemented; selected API/WebSocket integration checks were not rerun after the final UX/provider refinement.
Stage 2 - report/code	Evidence package, source manifest, tests, security and reflection.	Complete.
Stage 3 - complexity/relevance	useVisionAuto encapsulates reactive state, cancellable polling, stale-run protection, lifecycle cleanup and Leaflet synchronisation.	Advanced Vue technique; directly relevant and source/test evidenced.
Stage 3 - implementation	Run-local job identity, abort-aware delay, backend cancellation, adapter safety, dish-first and explicit uncertainty states.	Functional; two lifecycle defects fixed and reverified on revision commit.
Stage 3 - video/live coding	Demo script exists; recording is external to this report.	Requires final submitted video.

Table 13. Assessment rubric evidence matrix.

Appendix B - Technology Decision Register

Technology	Responsibility	Alternatives	Decision rationale
Vue 3 3.5.34	Frontend components and Composition API	React; Angular	Required by unit and scales from simple reactivity to composables.
Vite 8.1.4	Development/build	Vue CLI; webpack	Fast workflow and route code splitting.
Vue Router 5.0.7	Navigation and guards	Manual routing	Central role metadata and lazy views.
Pinia 3.0.4	Shared domain state	Vuex; component state	Cross-page and WebSocket reconciliation.
Bootstrap 5.3.8	Grid and utilities	Tailwind; pure CSS	Direct rubric compliance plus custom design.
Express 4.22.2	REST API and proxy	NestJS; FastAPI	Minimal JavaScript middleware model.
MySQL 8 / mysql2	Relational persistence	MongoDB; SQLite	Strong ownership and uniqueness constraints.
ws 8.21.0	Real-time recipe rooms	Polling; Socket.IO; SSE	Lightweight authenticated events.
Leaflet 1.9.4	Food Map	Mapbox GL; Google Maps JS	Open and direct marker/layer control.
FastAPI 0.136.3	Python AI service	Flask; Node-only	Typed validation and ML ecosystem access.
SentenceTransformer 5.5.1	Semantic embeddings	Hosted embedding API; TF-IDF	Local semantics without per-call provider cost.
OpenCLIP 3.3.0	Food/dish image-text matching	Custom CNN; hosted vision	Prompt-based international dish flexibility.
Tesseract.js 7.0.0	Local OCR	Google Vision; EasyOCR	No mandatory provider key and deterministic tests.
Guardian API	Current food news	NewsAPI; static JSON	Server-proxied current content.
Geoapify + OpenStreetMap	Address geocoding and serving-place alternatives	Google Places; local-only search	Provides provider-labelled candidates while retaining manual pinning, fallback behaviour and review-only uncertainty.

Table 14. Technology decision register.

Appendix C - Detailed Test and Reproducibility Record

The source identity is branch codex/render-deployment and commit 0473287788bad6a098f1d2724ec0e3d0c81bd412. The separately submitted source archive is FoodStory_Final_Source_04732877.zip: 55,496,926 bytes (52.925993 MiB), SHA-256 3d66a5f34ac2bbc854c0ce4104c89a85a523385b33216bf4cc60b5d0d2d23e1d, 814 entries, 767 files and 47 directories. ZIP integrity, every-entry reads, safe-path, forbidden/sensitive-file, likely-secret, clean extraction and expected-structure checks passed. The report-input DOCX was excluded from that archive. Final-commit verification records 48/48 frontend tests, a successful frontend production build, 399/399 backend tests and a clean Git worktree. Earlier focused suites are listed by concern and overlap with broader coverage, so their counts are not added together. The commands and final totals above are author-run local project evidence rather than independent CI certification.

Table 15. Detailed final test record.

Appendix D - Vision Auto Case and Failure Register

Case	Observed behaviour	Engineering significance
Positive local match	A public video case resolved to the stored Pho Le record with address and coordinates.	Demonstrates local match when evidence and catalogue agree.
Manual frame at ~53 s	Address visible in a late frame while an earlier pipeline returned empty OCR.	Demonstrates frame-selection recall failure.
Partial OCR	Street/district or place fragments without a complete verified address.	Retained as review evidence, not auto-resolved.
Heavy unresolved	Multiple fragments after OCR/ASR but no candidate passed the quality gate.	More text does not equal corroborated evidence.
No-evidence Short	Frames and ASR produced no useful address evidence; confidence remained zero.	Correct safe not-found outcome.
Provider unavailable	External serving-place search could not run.	Displayed as configuration/provider state, not “no restaurants”.
Mock/live confusion found	A historical script accepted a URL argument but ran mocked tests.	Report now labels fixture and live evidence explicitly.
Environment failures	yt-dlp/FFmpeg PATH issues and blocked service-account creation.	Native tools and cloud governance are real deployment dependencies.
Stale create-response race	A superseded create response could overwrite the shared active job ID after a newer run had started.	Run-local job IDs and stale-job cancellation now protect newer-run authority.
Polling timer after cancellation	The elapsed interval stopped, but the pending 1.5-second polling timeout remained queued.	The delay now listens to AbortController and clears immediately on cancel or unmount.

Table 16. Vision Auto case and failure register.

Appendix E - Screenshot Evidence Index

The body contains the strongest workflow figures. The complete historical screenshot package remains a separate evidence artefact. The final UX and provider figures in this edition were captured from the Render deployment and correspond to branch codex/render-deployment, commit 04732877. Historical provider-unavailable and two-browser states retain explicit evidence labels rather than being presented as current provider success.

Selected Supplemental Evidence

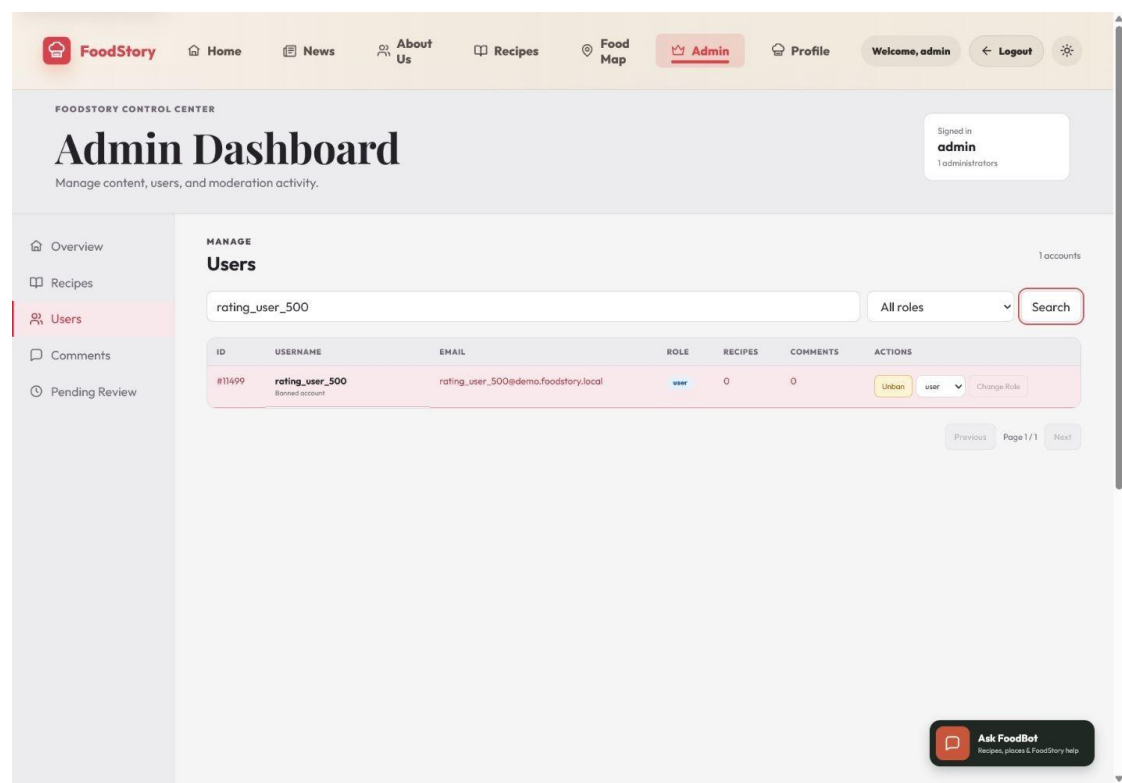


Figure 23. Administrator governance view with searchable users, role controls and account-status actions; no destructive governance action was executed for this capture.

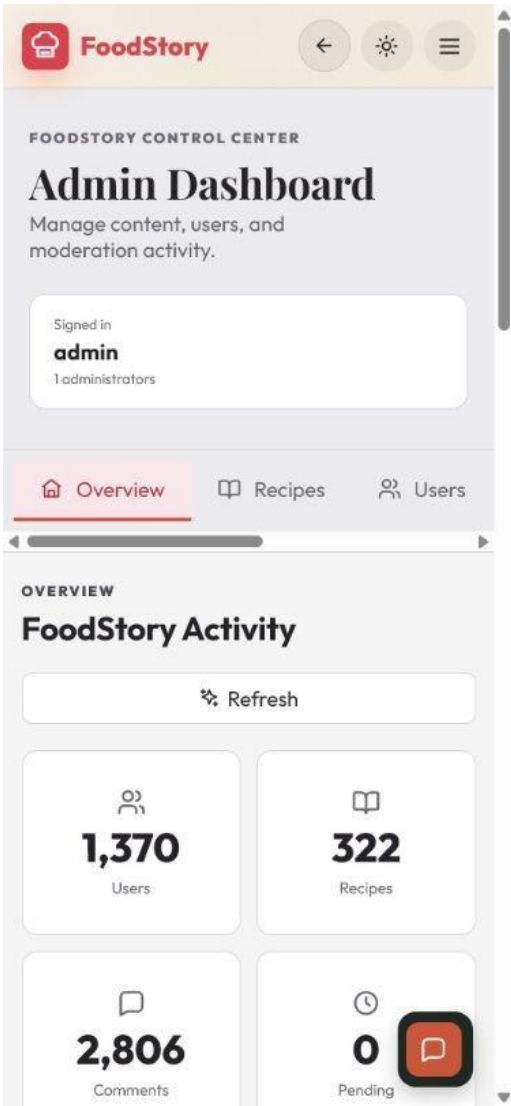


Figure 24. Mobile Admin overview at the 390 x 844 target without document-level horizontal overflow.

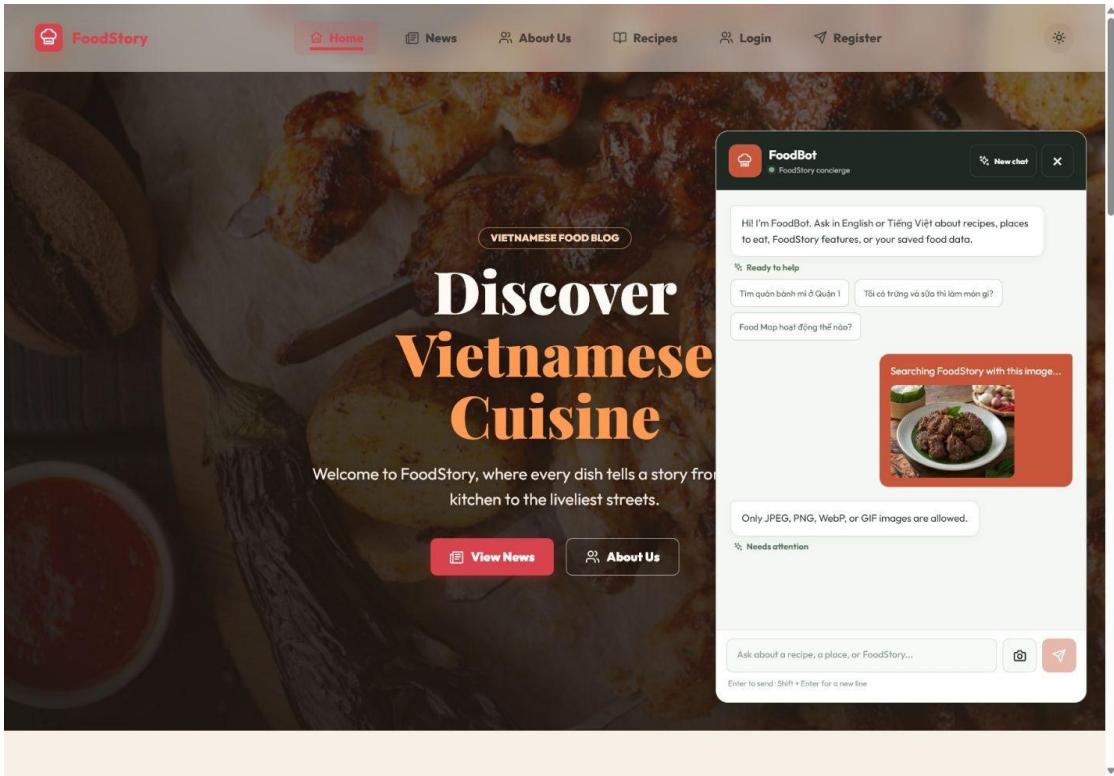


Figure 25. Genuine FoodBot image-upload validation failure in the browser-automation path; retained as limitation evidence.

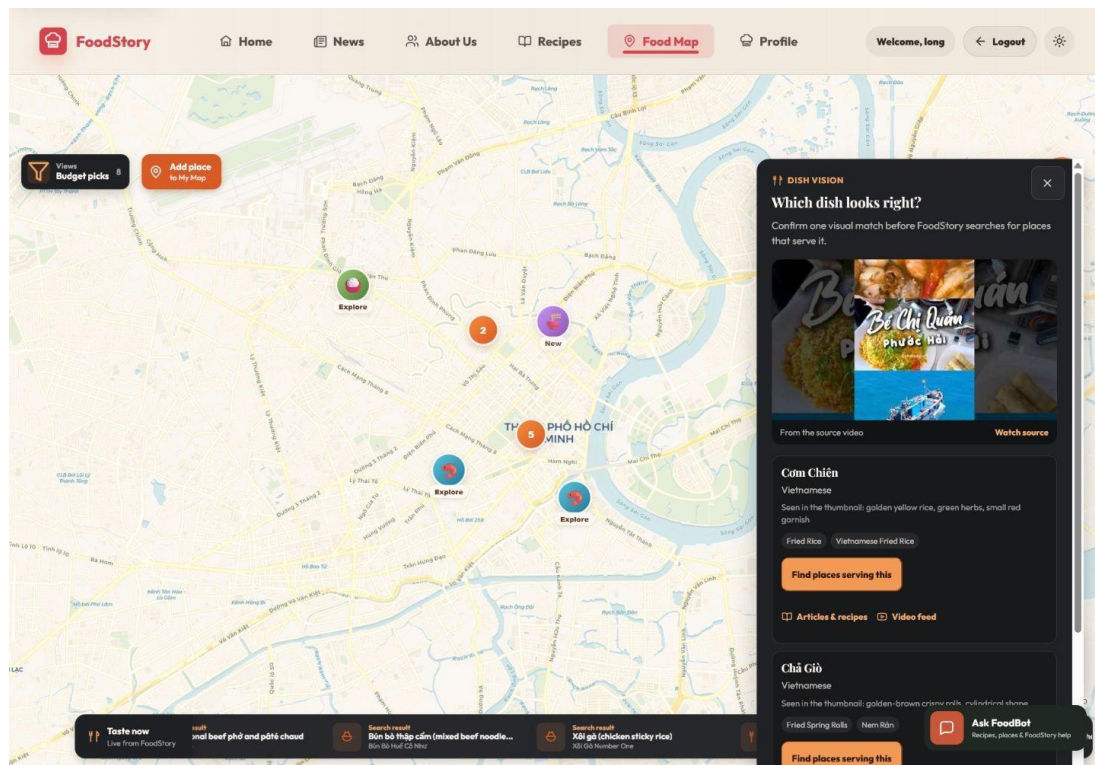


Figure 26. Dish-first confirmation presents bounded visual suggestions before place search and preserves uncertainty about the filming location.

Appendix F - Final Source and Submission Checklist

- Final branch: codex/render-deployment.
- Final commit: 047328778bad6a098f1d2724ec0e3d0c81bd412.
- Final archive: FoodStory_Final_Source_04732877.zip; 55,496,926 bytes (52.925993 MiB); SHA-256 3d66a5f34ac2bbc854c0ce4104c89a85a523385b33216bf4cc60b5d0d2d23e1d; 814 entries, 767 files and 47 directories.
Archive checks completed: integrity, every-entry read, safe paths, report input exclusion, forbidden/sensitive-file scan, likely-secret scan, clean extraction and expected project structure - PASS.
- Real .env and service-account files must be excluded; sanitised examples may remain. Any previously exposed provider keys should be rotated.
- Final verification: frontend npm ci passed; frontend build passed; frontend tests 48/48; backend npm ci passed; backend tests 399/399; Git worktree clean.
- Evidence statuses follow the interpretation guide in Chapter 6; historical, deterministic, provider-limited and live observations remain distinguishable.
- Render deployment URL: <https://foodstory-cos30043-hoanglong212.onrender.com>. Stage 3 video URL: [STAGE_3_VIDEO_URL_TO_BE_INSERTED].
- The source submission archive was generated from commit 04732877 with the report-input DOCX excluded. The archive and report are separate deliverables. This archive-verified report remains video-incomplete until [STAGE_3_VIDEO_URL_TO_BE_INSERTED] is replaced.
- Final UX/provider browser evidence was captured from the Render deployment on 27 July 2026 and corresponds to branch codex/render-deployment, commit 04732877. Historical states remain labelled and were not fabricated.